

- Kryptographisch sichere **Pseudozufallszahlengeneratoren**
Cryptographic Secure Pseudo Random Number Generators (CSPRNG)
- Kryptographische **Hashfunktionen**
Message Digests (Cryptographic Hash Functions)
- **Symmetrische Verschlüsselungsverfahren** (Symmetric Ciphers)
- **Message Authentication Codes** (MACs)
- **Asymmetrische Verschlüsselungsverfahren** (Asymmetric Ciphers)
- **Digitale Signaturen** (Digital Signatures)
- **Schlüsselaustauschverfahren** (Key Exchange / Key Establishment)

- Security Developer's Guide
 - Java Cryptography Architecture (JCA)
 - JCA Provider (JDK Providers)
 - SUN Provider, SunRsaSign Provider
 - SunJCE Provider
 - SunJSSE Provider
 - SunPCSC Provider
 - ...

Provider der ersten Stunde

Java Cryptographic Extension

Java Secure Socket Extension (TLS)

SmartCard API

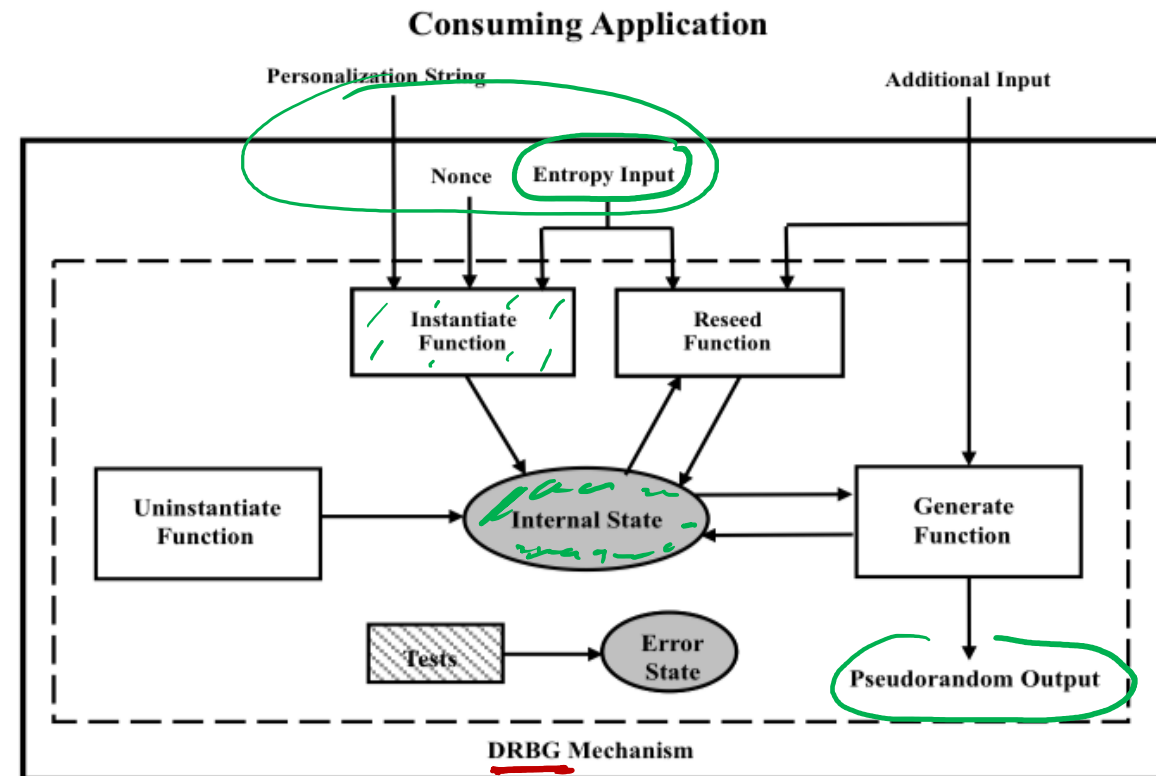
```
31  
32 ☐ public static void providerDemo() {  
33     Provider[] providers = Security.getProviders();  
34     int ctr = 1;  
35     ☐ for (Provider p : providers) {  
36         System.out.printf("%s\n%d.) %s\n%s\n",  
37             "=".repeat(80), ctr++, p, p.getInfo());  
38  
39         Set<Map.Entry<Object, Object>> ents = p.entrySet();  
40         ☐ for (Map.Entry ent : ents) {  
41             System.out.println(ent);  
42         }  
43     }  
44 }
```

- Kryptographisch sichere
Pseudozufallszahlengeneratoren
- Kryptographische **Hashfunktionen**
- **Symmet. Verschlüsselungsverfahren**
- **Message Authentication Codes**
- **Asymmet. Verschlüsselungsverfahren**
- **Digitale Signaturen**
- **Schlüsselaustauschverfahren**
- [java.security.SecureRandom](#)
- [java.security.MessageDigest](#)
- [javax.crypto.Cipher](#)
- [javax.crypto.Mac](#)
- [javax.crypto.Cipher](#)
- [java.security.Signature](#)
- [javax.crypto.KeyAgreement](#)

- JCA -Implementierungen: SecureRandom
- Recommendation for Random Number Generation Using Deterministic Random Bit Generators, NIST Special Publication 800-90A, June 2015

<http://dx.doi.org/10.6028/NIST.SP.800-90Ar1>

- DRBG Functional Model:



10.1	DRBG Mechanisms Based on Hash Functions	37
10.1.1	<u>Hash</u> _DRBG	38
10.1.1.1	Hash_DRBG Internal State	39
10.1.1.2	Instantiation of Hash_DRBG	39
10.1.1.3	Reseeding a Hash_DRBG Instantiation	40
10.1.1.4	Generating Pseudorandom Bits Using Hash_DRBG	41
10.1.2	<u>HMAC</u> _DRBG	43
10.1.2.1	HMAC_DRBG Internal State	43
10.1.2.2	The HMAC_DRBG Update Function (Update).....	44
10.1.2.3	Instantiation of HMAC_DRBG	45
10.1.2.4	Reseeding an HMAC_DRBG Instantiation	46
10.1.2.5	Generating Pseudorandom Bits Using HMAC_DRBG	46
10.2	DRBG Mechanism Based on Block Ciphers	48
10.2.1	<u>CTR</u> _DRBG	48
10.2.1.1	CTR_DRBG Internal State	50
10.2.1.2	The Update Function (CTR_DRBG_Update)	51
10.2.1.3	Instantiation of CTR_DRBG	52
10.2.1.4	Reseeding a CTR_DRBG Instantiation	54
10.2.1.5	Generating Pseudorandom Bits Using CTR_DRBG	55

CTR-Mode ein
Blockverschlüsselungsalgo. (AES)

10.1 DRBG Mechanisms Based on Hash Functions 37

10.1.1 Hash_DRBG 38

	SHA-1	SHA-224 and SHA-512/224	SHA-256 and SHA-512/256	SHA-384	SHA-512
Supported security strengths	See [SP 800-57]				
<i>highest_supported_security_strength</i>	See [SP 800-57]				
Output Block Length (<i>outlen</i>)	160	224	256	384	512
Required minimum entropy for instantiate and reseed	<i>security_strength</i>				
Minimum entropy input length (<i>min_length</i>)	<i>security_strength</i>				
Maximum entropy input length (<i>max_length</i>)	2^{35} bits				
Seed length (<i>seedlen</i>) for Hash_DRBG	440	440	440	888	888
Maximum personalization string length (<i>max_personalization_string_length</i>)	2^{35} bits				
Maximum additional_input length (<i>max_additional_input_length</i>)	2^{35} bits				
<i>max_number_of_bits_per_request</i>	2^{19} bits				
Maximum number of requests between reseeds (<i>reseed_interval</i>)	2^{48}				

10.1.1	Hash_DRBG	38
10.1.1.1	Hash_DRBG Internal State	39
10.1.1.2	Instantiation of Hash_DRBG	39
10.1.1.3	Reseeding a Hash_DRBG Instantiation	40
10.1.1.4	Generating Pseudorandom Bits Using Hash_DRBG	41

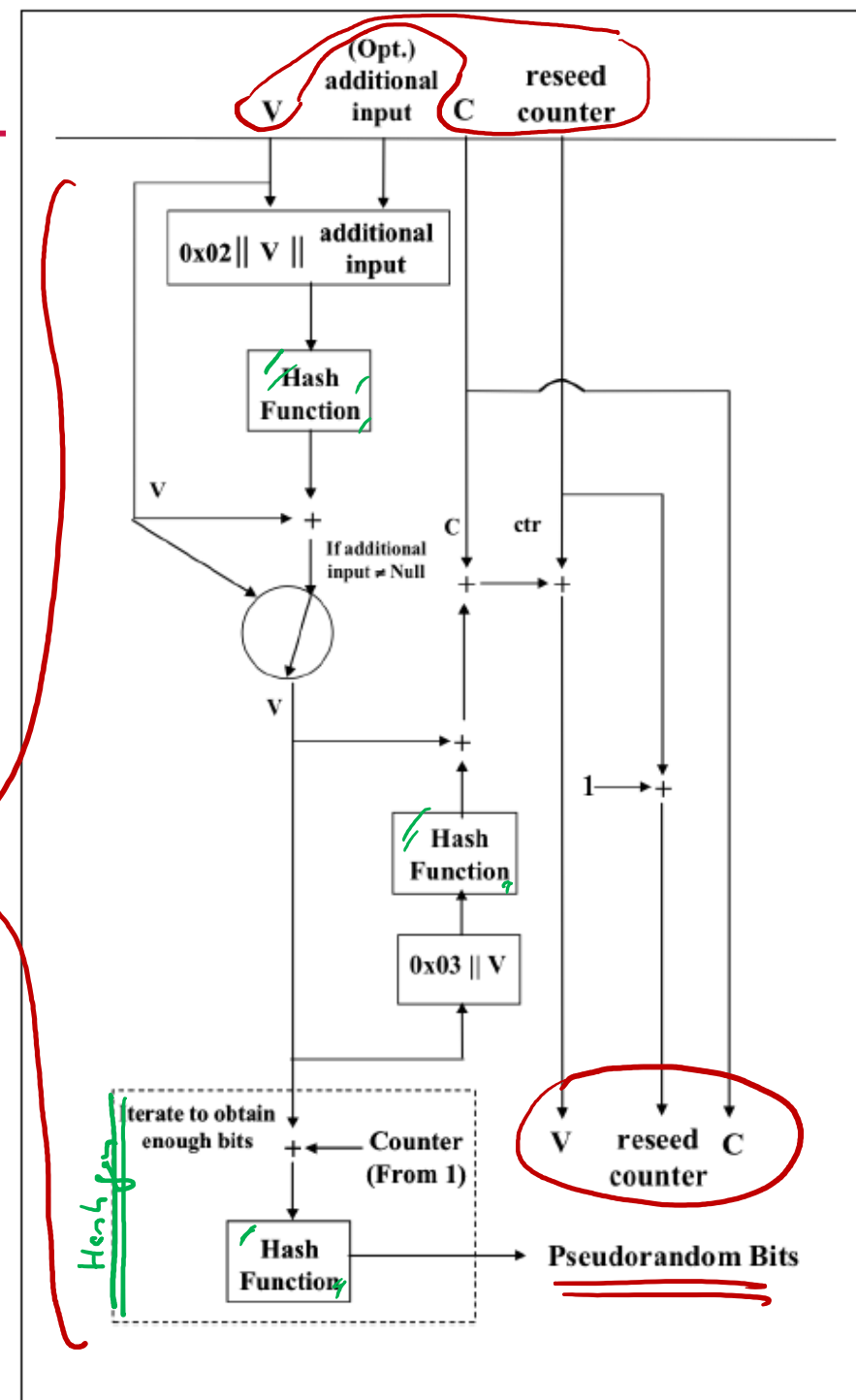
Hash_DRBG Instantiate Process:

1. $seed_material = entropy_input \parallel nonce \parallel personalization_string$.
2. $seed = \mathbf{Hash_df}(seed_material, seedlen)$.
3. $V = seed$.
4. $C = \mathbf{Hash_df}((0x00 \parallel V), seedlen)$. Comment: Precede V with a byte of zeros.
5. $reseed_counter = 1$.
6. **Return** $(V, C, reseed_counter)$.

10.1.1	Hash_DRBG	38
10.1.1.1	Hash_DRBG Internal State	39
10.1.1.2	Instantiation of Hash_DRBG	39
10.1.1.3	Reseeding a Hash_DRBG Instantiation	40
10.1.1.4	Generating Pseudorandom Bits Using Hash_DRBG	41

Hash_DRBG_Generate Process:

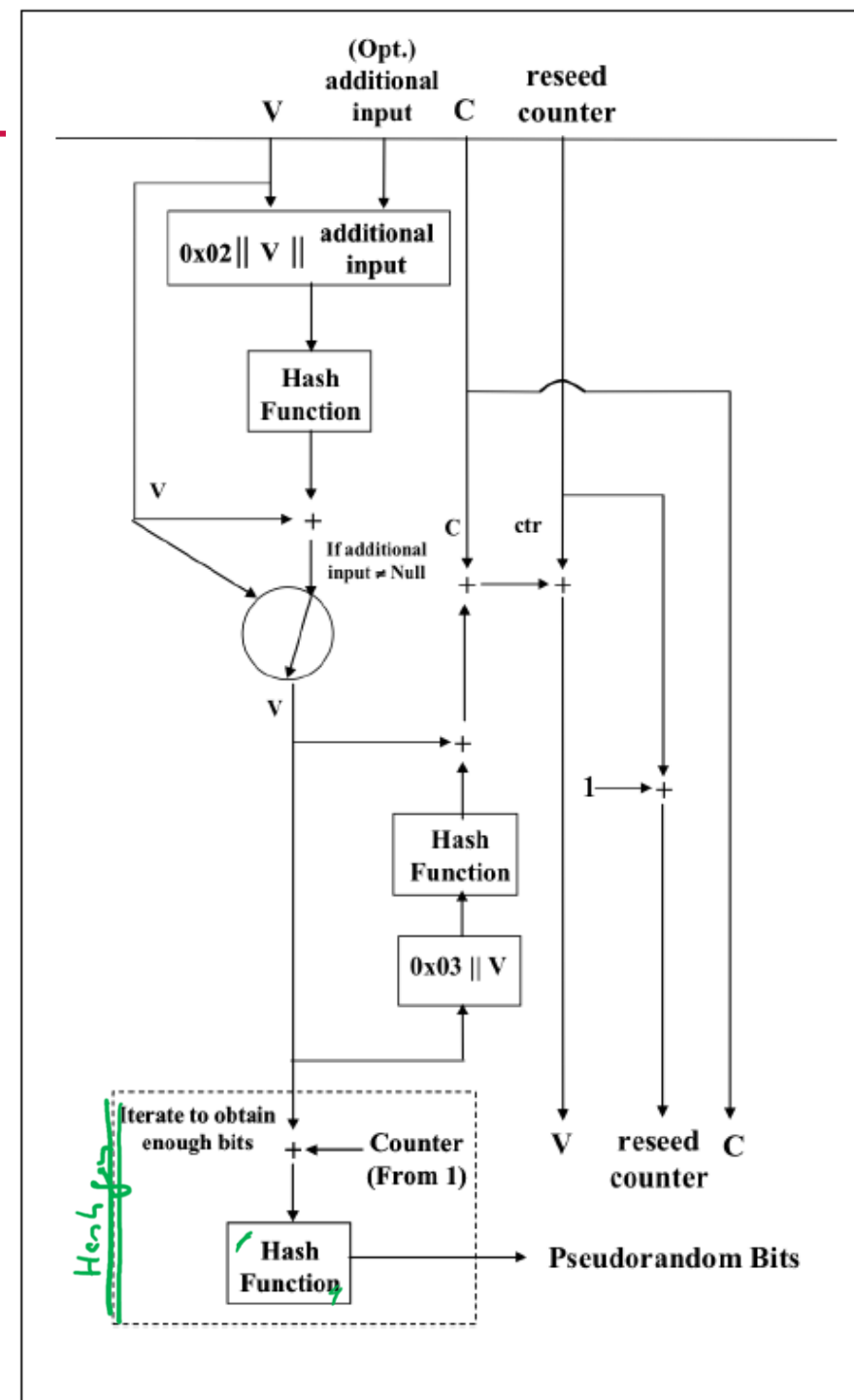
1. If $reseed_counter > reseed_interval$, then return an indication that a reseed is required.
2. If ($additional_input \neq Null$), then do
 - 2.1 $w = \mathbf{Hash}(0x02 \parallel V \parallel additional_input)$.
 - 2.2 $V = (V + w) \bmod 2^{seedlen}$.
3. ($returned_bits$) = **Hashgen**($requested_number_of_bits, V$).
4. $H = \mathbf{Hash}(0x03 \parallel V)$.
5. $V = (V + H + C + reseed_counter) \bmod 2^{seedlen}$.
6. $reseed_counter = reseed_counter + 1$.
7. Return (**SUCCESS**, $returned_bits$, V , C , $reseed_counter$).



10.1.1	Hash_DRBG	38
10.1.1.1	Hash_DRBG Internal State	39
10.1.1.2	Instantiation of Hash_DRBG	39
10.1.1.3	Reseeding a Hash_DRBG Instantiation	40
10.1.1.4	Generating Pseudorandom Bits Using Hash_DRBG	41

Hashgen Process:

1. $m = \left\lceil \frac{\text{requested_no_of_bits}}{\text{outlen}} \right\rceil$.
2. $\text{data} = V$.
3. $W = \text{the Null string}$.
4. For $i = 1$ to m
 - 4.1 $w = \text{Hash}(\text{data})$.
 - 4.2 $W = W \parallel w$.
 - 4.3 $\text{data} = (\text{data} + 1) \bmod 2^{\text{seedlen}}$.
5. $\text{returned_bits} = \text{leftmost}(W, \text{requested_no_of_bits})$.
6. Return (returned_bits).



```
10
11 [-] public static void main(String[] args) throws Exception {
12     demo01();
13     // demo_setSeed();
14 }
15
16 [-] public static void demo01() throws Exception {
17     SecureRandom rng = new SecureRandom();
18     System.out.println("Random int: " + rng.nextInt());
19     // getPrngInfo(rng);
20 }
21
22 [-] public static void getPrngInfo(SecureRandom csprng) {
23     System.out.println("-".repeat(60));
24     System.out.println("csprng.toString(): " + csprng);
25     System.out.println("csprng.getAlgorithm(): " + csprng.getAlgorithm());
26     System.out.println("csprng.getParameters(): " + csprng.getParameters());
27 }
```

```
95  
96 ☐ public static void demo_setSeed() throws NoSuchAlgorithmException {  
97     SecureRandom csprng = new SecureRandom();  
98     csprng.setSeed(42);  
99     System.out.printf("csprng.nextInt():      %08x\n", csprng.nextInt());  
100 }  
101
```

GEFÄHRLICH !!!

- ✓ ■ Kryptographisch sichere **Pseudozufallszahlengeneratoren**
Cryptographic Secure Pseudo Random Number Generators (CSPRNG)
- Kryptographische **Hashfunktionen**
Message Digests (Cryptographic Hash Functions)
- ■ **Symmetrische Verschlüsselungsverfahren** (Symmetric Ciphers)
- **Message Authentication Codes** (MACs)
- **Asymmetrische Verschlüsselungsverfahren** (Asymmetric Ciphers)
- **Digitale Signaturen** (Digital Signatures)
- **Schlüsselaustauschverfahren** (Key Exchange / Key Establishment)

} ohne Schlüssel

}  secret key

}  private key
 public key

- ① Stromverschlüsselungsverfahren (Stream Cipher)
- ② Blockverschlüsselungsverfahren (Block Cipher)
- 3 Mathematischer Exkurs: Endliche Körper $\mathbb{F}_{p^n}$
- 4 Hashfunktionen
- 5 MAC-Verfahren

Wichtigste Beispiele

ChaCha20

AES

TLS-Algo

Stromverschlüsselungsverfahren (Stream Cipher)

1.1	Grundlagen	2
1.2	Schlüsselstrom vs. One-Time-Pad	8
1.3	Nonces zur Initialisierung eines Schlüsselstromgenerators	9
1.4	ChaCha20	10
1.5	Cipher-Instanzen: Verschlüsselungsalgorithmen in Java- Laufzeitumgebungen	18

Stromverschlüsselungsverfahren (Stream Cipher)

Bei einem Stromverschlüsselungsverfahren wird eine Nachricht zeichenweise verschlüsselt. Die Menge der Nachrichten $\mathcal{M}$ ist dabei durch die Menge der endlichen Folgen von Elementen aus einer Zeichenmenge $\mathcal{Z}$ definiert:

$$\mathcal{M} = \mathcal{Z}^{>0} := \bigcup_{l \in \mathbb{N}} \mathcal{Z}^l = \{(m_1, m_2, \dots, m_l) \mid m_i \in \mathcal{Z}, l \in \mathbb{N}\}$$

$$\mathcal{Z} = \mathbb{Z}_{26}, \mathbb{Z}_2, (\mathbb{Z}_2)^p$$

Eine Verschlüsselungsfunktion E_k eines Stromverschlüsselungsverfahrens überführt Folgen der Länge l in Folgen gleicher Länge l (für alle $l \in \mathbb{N}$) und lässt sich darüber hinaus zeichenweise beschreiben, d. h., es gibt Funktionen

$$E^{(i)} : \mathcal{K} \times \mathcal{Z}^i \rightarrow \mathcal{Z} \quad \text{für alle } i \in \mathbb{N}$$

mit:

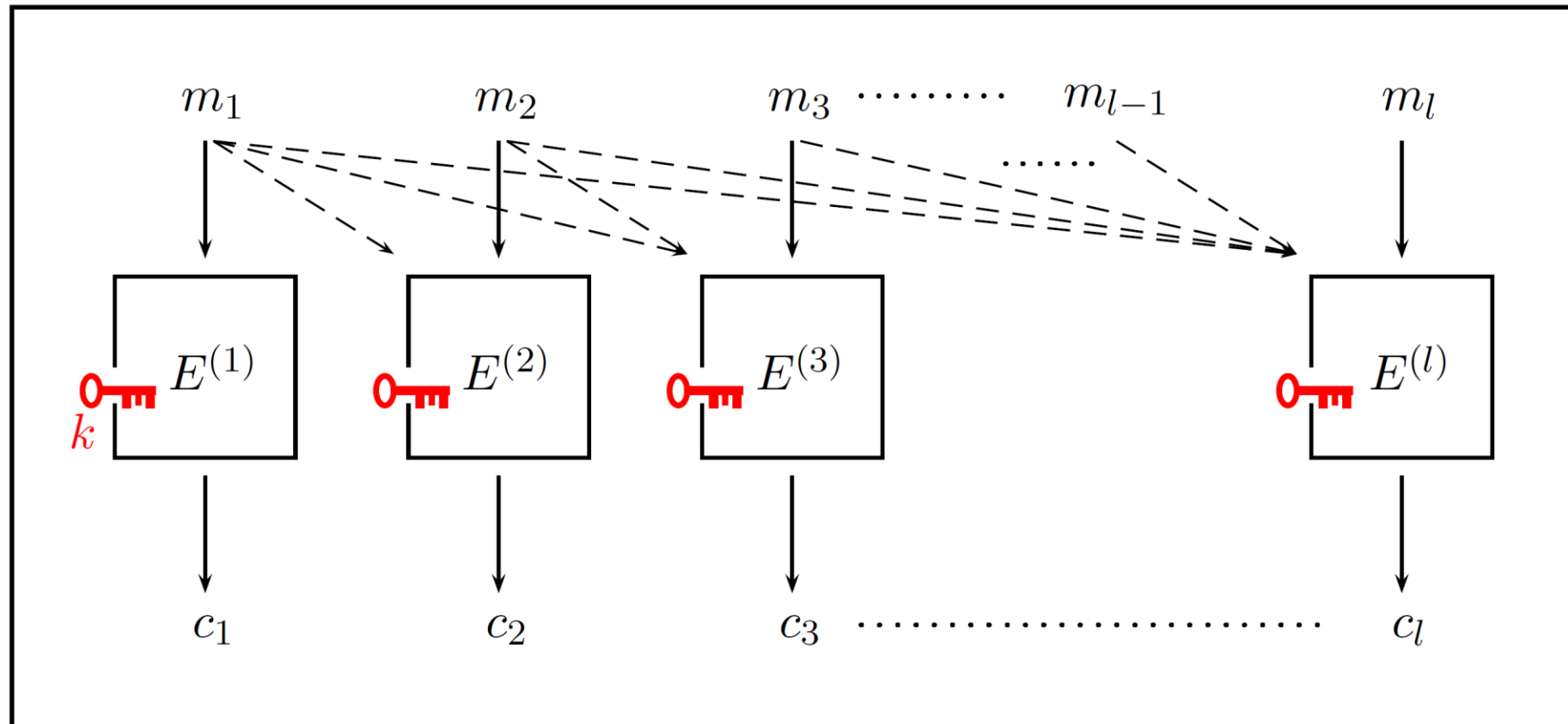
$$\begin{aligned} E_k((m_1, m_2, \dots, m_l)) \\ = \left(E^{(1)}(k, (m_1)), E^{(2)}(k, (m_1, m_2)), \dots, E^{(l)}(k, (m_1, m_2, \dots, m_l)) \right) \end{aligned}$$

In die Berechnung des i -ten Zeichens des Geheimtextes

$$c_i = E^{(i)}(k, (m_1, m_2, \dots, m_i))$$

können neben dem Schlüssel und dem i -ten Zeichen des Klartextes auch die vorherigen Zeichen einfließen. Die nachfolgenden Zeichen und die Gesamtlänge der Nachricht sind aber ohne Belang.

Bezüglich der Variablen m_i muss $E^{(i)}$ eine bijektive Funktion sein, deren Inverse zur Entschlüsselung benötigt wird.



$$\mathcal{M} = \mathcal{Z}^{>0}$$

$$E^{(i)} : \mathcal{K} \times \mathcal{Z}^i \rightarrow \mathcal{Z}$$

$$c_i = E^{(i)}(k, (m_1, m_2, \dots, m_i))$$

Abb. 1.1 Stromverschlüsselungsverfahren

$$\begin{aligned} E_k((m_1, m_2, \dots, m_l)) \\ = (E^{(1)}(k, (m_1)), E^{(2)}(k, (m_1, m_2)), \dots, E^{(l)}(k, (m_1, m_2, \dots, m_l))) \end{aligned}$$

Beispiel 1.1: Monoalphabetisches Verschlüsselungsverfahren

Das monoalphabetische Verschlüsselungsverfahren (vgl. mit Studienheft 01, Beispiel 1.4) ist ein Stromverschlüsselungsverfahren mit:

$$\mathcal{L} = \mathbb{Z}_{26}$$

$$\mathcal{K} = \Sigma_{\mathbb{Z}_{26}} = \{k : \mathbb{Z}_{26} \rightarrow \mathbb{Z}_{26} \mid k \text{ ist bijektiv}\}$$

$$c_i = E^{(i)}(k, (m_1, m_2, \dots, m_i)) = k(m_i)$$

Zur Berechnung des Geheimtextzeichens c_i wird neben dem Schlüssel k nur das entsprechende Klartextzeichen m_i benötigt. Das monoalphabetische Verschlüsselungsverfahren ist daher ein Beispiel für ein synchrones Stromverschlüsselungsverfahren entsprechend der folgenden Definition. 

Synchrone Stromverschlüsselungsverfahren (Synchronous Stream Cipher)

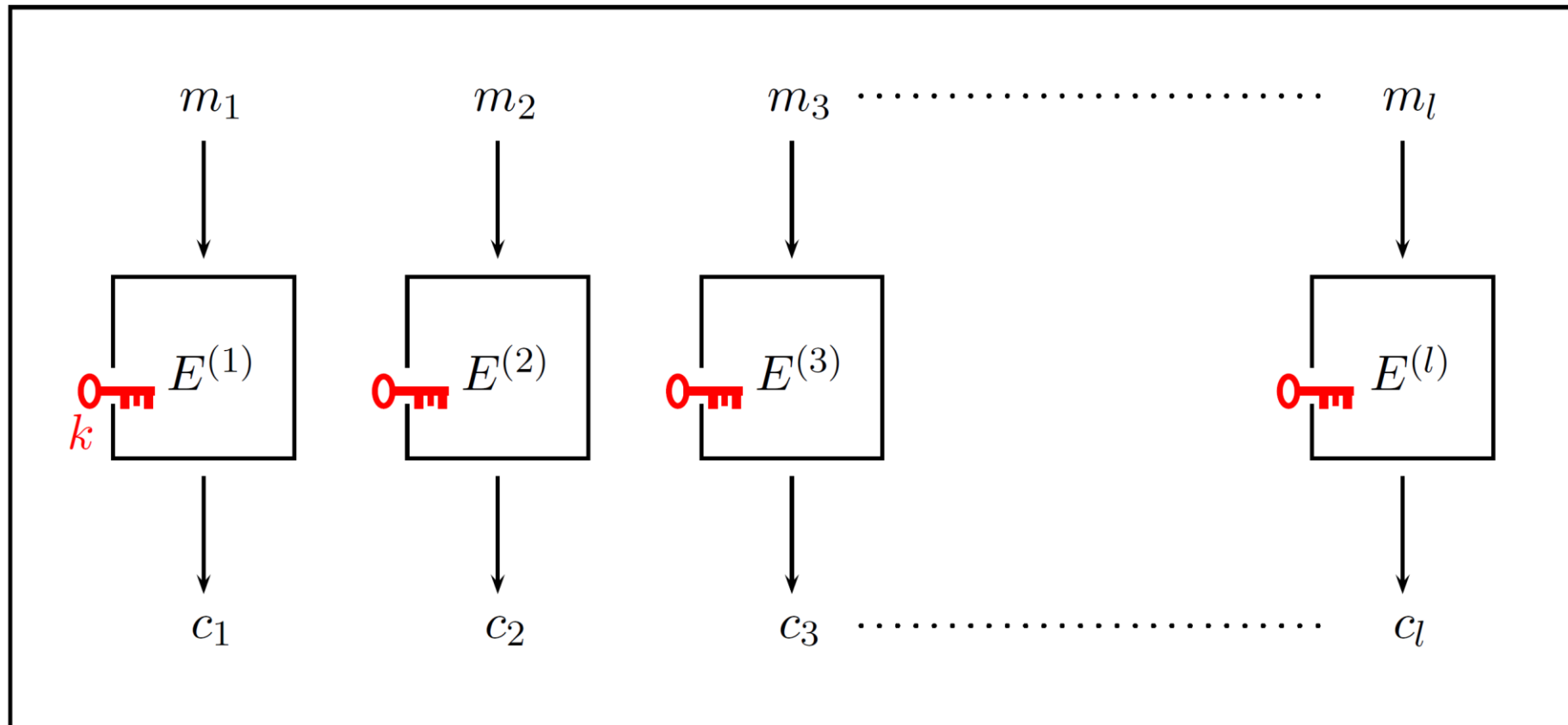
Ist die Verschlüsselung des i -ten Zeichens für jedes $i \in \mathbb{N}$ durch eine Funktion

$$E^{(i)} : \mathcal{K} \times \mathcal{Z} \rightarrow \mathcal{Z}$$

bestimmt, so wird das Stromverschlüsselungsverfahren *synchron* genannt.

Es gilt dann:

$$E_k((m_1, m_2, \dots, m_l)) = (E^{(1)}(k, m_1), E^{(2)}(k, m_2), \dots, E^{(l)}(k, m_l))$$



$$\mathcal{M} = \mathcal{Z}^{>0}$$

$$E^{(i)} : \mathcal{K} \times \mathcal{Z} \rightarrow \mathcal{Z}$$

$$c_i = E^{(i)}(k, m_i)$$

Abb. 1.2 Synchrones Stromverschlüsselungsverfahren

$$E_k((m_1, m_2, \dots, m_l)) = (E^{(1)}(k, m_1), E^{(2)}(k, m_2), \dots, E^{(l)}(k, m_l))$$

Beispiel 1.2: Monoalphabetisches Verschlüsselungsverfahren

Wie bereits in Beispiel 1.1 erwähnt ist das monoalphabetische Verschlüsselungsverfahren ein synchrones Stromverschlüsselungsverfahren. Die in der Beschreibung von synchronen Stromverschlüsselungsverfahren auftretenden Funktionen $E^{(i)}$ sind in diesem Fall für alle $i \in \mathbb{N}$ gleich:

$$E^{(i)} : \mathcal{K} \times \mathcal{Z} \rightarrow \mathcal{Z}, \quad E^{(i)}(k, m_i) = k(m_i) \quad \left(\mathcal{Z} = \mathbb{Z}_{26}, \mathcal{K} = \Sigma_{\mathbb{Z}_{26}} \right)$$

Frage 1

Das in Studienheft 01, Beispiel 1.4 beschriebene polyalphabetische Verschlüsselungsverfahren ist ein synchrones Stromverschlüsselungsverfahren. Finden Sie nach einer Identifikation der Buchstabenmenge $\{A, B, \dots, Z\}$ mit $\mathbb{Z}_{26}$ die Abbildungsvorschrift für $E^{(i)}$ für $i \in \mathbb{N}$.

Ilies-Test

Zustandsabhängige Stromverschlüsselungsverfahren

Gegeben seien eine endliche Menge $\mathcal{S}$ von Zuständen sowie Abbildungen:

$$s_0 : \mathcal{K} \rightarrow \mathcal{S}$$

$$s : \mathcal{S} \times \mathcal{Z} \rightarrow \mathcal{S}$$

$$E : \mathcal{S} \times \mathcal{Z} \rightarrow \mathcal{Z}$$

Vor der Verschlüsselung des ersten Zeichens m_1 mit der Verschlüsselungsfunktion E wird in Abhängigkeit vom Schlüsselwert k mit s_0 ein Startzustand festgelegt. Vor der Verschlüsselung jedes weiteren Zeichens mit der Verschlüsselungsfunktion E wird mittels s jeweils eine Zustandsänderung bewirkt. Die Verschlüsselung von $(m_1, m_2, \dots, m_l)$ zu $(c_1, c_2, \dots, c_l)$ wird somit durch folgende Rekursion beschrieben:

Zustandsübergang

$$s_0 : \mathcal{K} \rightarrow \mathcal{S}$$

Initialisierung

$$s : \mathcal{S} \times \mathcal{Z} \rightarrow \mathcal{S}$$

Update-Funktion

$$E : \mathcal{S} \times \mathcal{Z} \rightarrow \mathcal{Z}$$

Verschlüsselung eines Zeichens

$$\sigma_1 := s_0(k)$$

$$c_1 := E(\sigma_1, m_1)$$

$$\sigma_2 := s(\sigma_1, m_1)$$

$$c_2 := E(\sigma_2, m_2)$$

$$\sigma_3 := s(\sigma_2, m_2)$$

$$c_3 := E(\sigma_3, m_3)$$

$\vdots$

$\vdots$

$$\sigma_l := s(\sigma_{l-1}, m_{l-1})$$

$$c_l := E(\sigma_l, m_l)$$

Für jeden festen Wert σ in der ersten Variablen muss E bezüglich der zweiten Variablen $m \in \mathcal{Z}$ eine bijektive Funktion sein, deren Inverse zur Entschlüsselung benötigt wird.

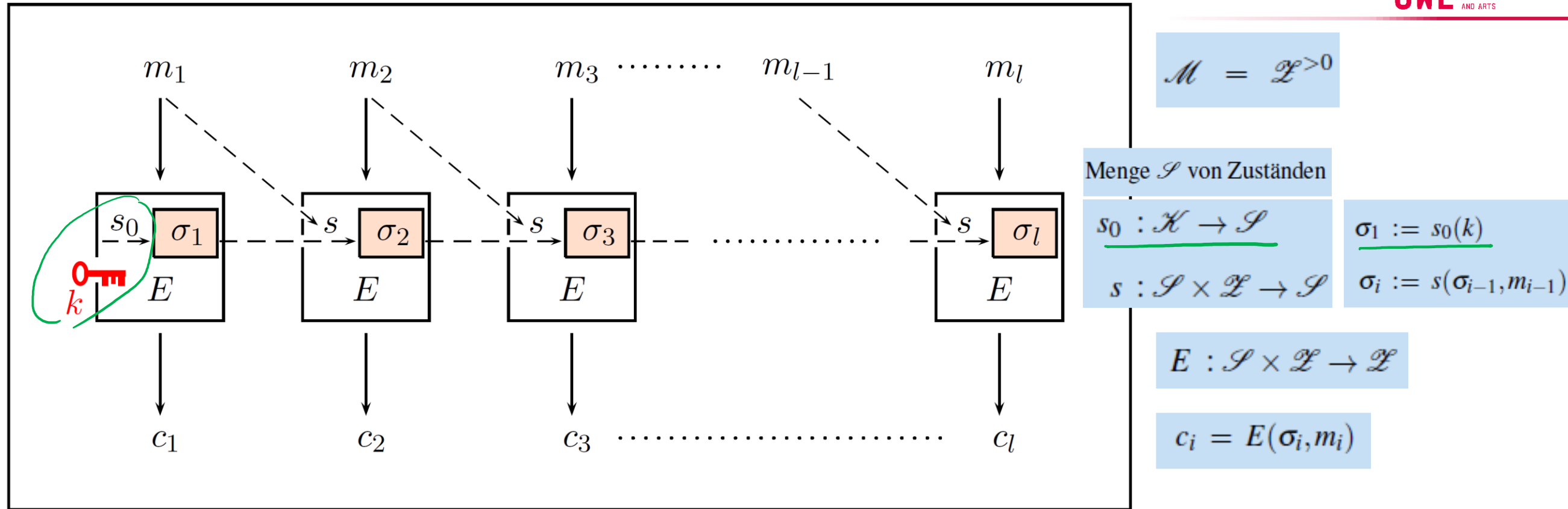


Abb. 1.3 Zustandsabhängiges Stromverschlüsselungsverfahren

Additive zustandsabhängige Stromverschlüsselungsverfahren

Zustandsabhängige Stromverschlüsselungsverfahren für die es eine Funktion

$$S : \mathcal{S} \rightarrow \mathcal{Z}$$

gibt, so dass für die Verschlüsselungsfunktion

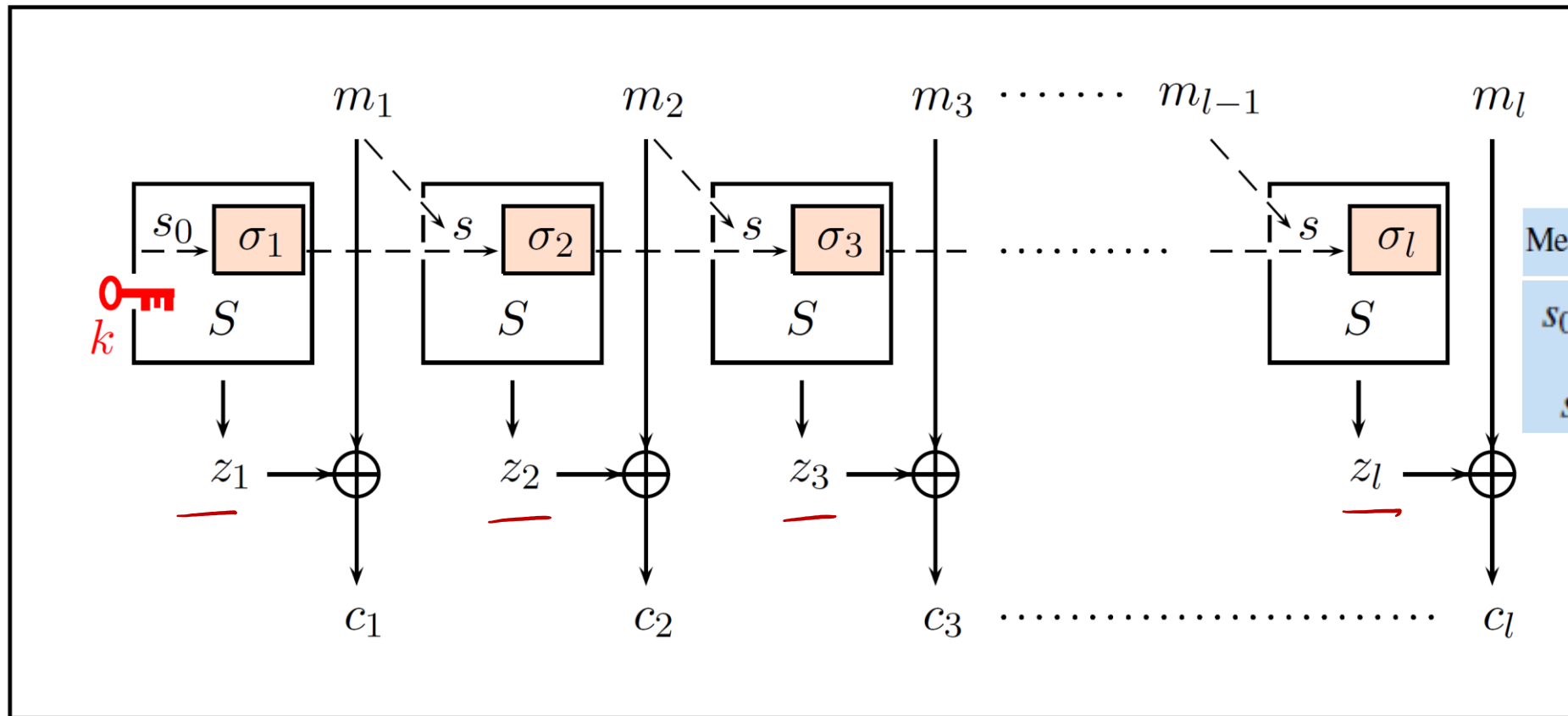
$$E(\sigma, m) = S(\sigma) + m$$

für alle $\sigma \in \mathcal{S}$ und $m \in \mathcal{Z}$ gilt, wird *additiv* genannt.

Die Funktionen s_0 , s und S bilden zusammen einen *Schlüsselstromgenerator* und die Folge der Zeichen

$$z_i := S(\sigma_i) = S(s(\sigma_{i-1}, m_{i-1}))$$

bilden den durch k und die Nachrichtenzeichen $m_1, m_2, \dots$ definierten *Schlüsselstrom*.



$$\mathcal{M} = \mathcal{Z}^{>0}$$

Menge $\mathcal{S}$ von Zuständen

$$s_0 : \mathcal{K} \rightarrow \mathcal{S}$$

$$\sigma_1 := s_0(k)$$

$$s : \mathcal{S} \times \mathcal{Z} \rightarrow \mathcal{S}$$

$$\sigma_i := s(\sigma_{i-1}, m_{i-1})$$

$$S : \mathcal{S} \rightarrow \mathcal{Z}$$

$$c_i = E(\sigma_i, m_i) = S(\sigma_i) + m_i$$

Abb. 1.4 Additive zustandsabhängige Stromverschlüsselungsverfahren

Die Funktionen s_0 , s und S bilden zusammen einen Schlüsselstromgenerator und die Folge der Zeichen

$$z_i := S(\sigma_i) = S(s(\sigma_{i-1}, m_{i-1}))$$

bilden den durch k und die Nachrichtenzeichen $m_1, m_2, \dots$ definierten Schlüsselstrom.

$$(z_1, z_2, \dots, z_l)$$

Synchrone additive Stromverschlüsselungsverfahren

Ein additives zustandsabhängiges Stromverschlüsselungsverfahren wird als *synchrone* additives Stromverschlüsselungsverfahren bezeichnet, falls die Übergangsfunktion nicht von den Zeichen der Klartextnachricht abhängt, d. h., falls gilt:

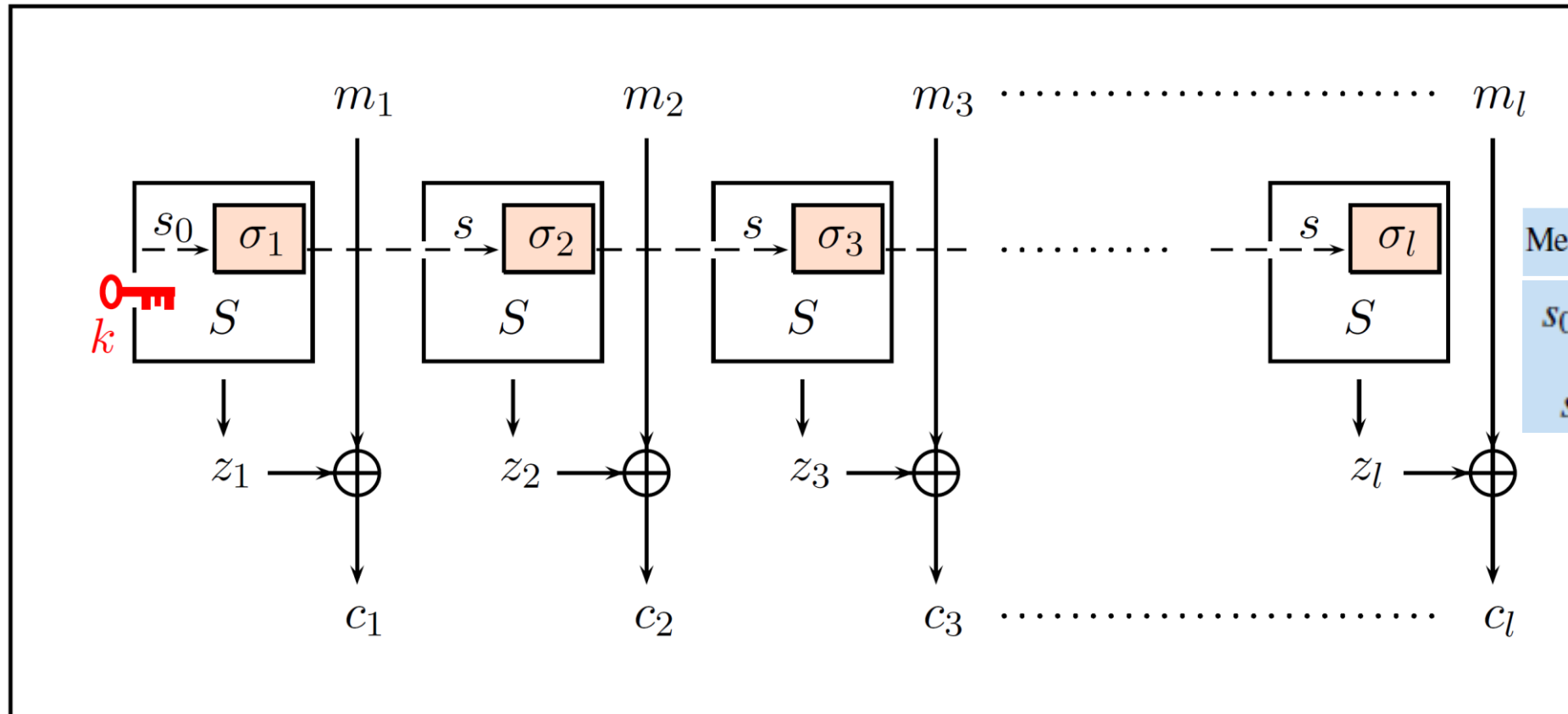
$$\sigma_{i+1} = s(\sigma_i) \quad \text{für alle } i \in \mathbb{N}$$

mit einer Funktion

$$s : \mathcal{S} \rightarrow \mathcal{S}$$

Der durch die Funktionen s_0 , s und S definierte Schlüsselstromgenerator erzeugt den Schlüsselstrom:

$$z_i = S(s^{i-1}(s_0(k))) \quad (i \in \mathbb{N})$$



$$\mathcal{M} = \mathbb{Z}^{>0}$$

Menge $\mathcal{S}$ von Zuständen

$$s_0 : \mathcal{K} \rightarrow \mathcal{S}$$

$$\sigma_1 := s_0(k)$$

$$s : \mathcal{S} \times \mathbb{Z} \rightarrow \mathcal{S}$$

$$\sigma_{i+1} = s(\sigma_i)$$

$$S : \mathcal{S} \rightarrow \mathbb{Z}$$

$$c_i = E(\sigma_i, m_i) = S(\sigma_i) + m_i$$

Abb. 1.5 Synchrone additive Stromverschlüsselungsverfahren

Der durch die Funktionen s_0 , s und S definierte Schlüsselstromgenerator erzeugt den Schlüsselstrom:

$$z_i = S(s^{i-1}(s_0(k))) \quad (i \in \mathbb{N})$$

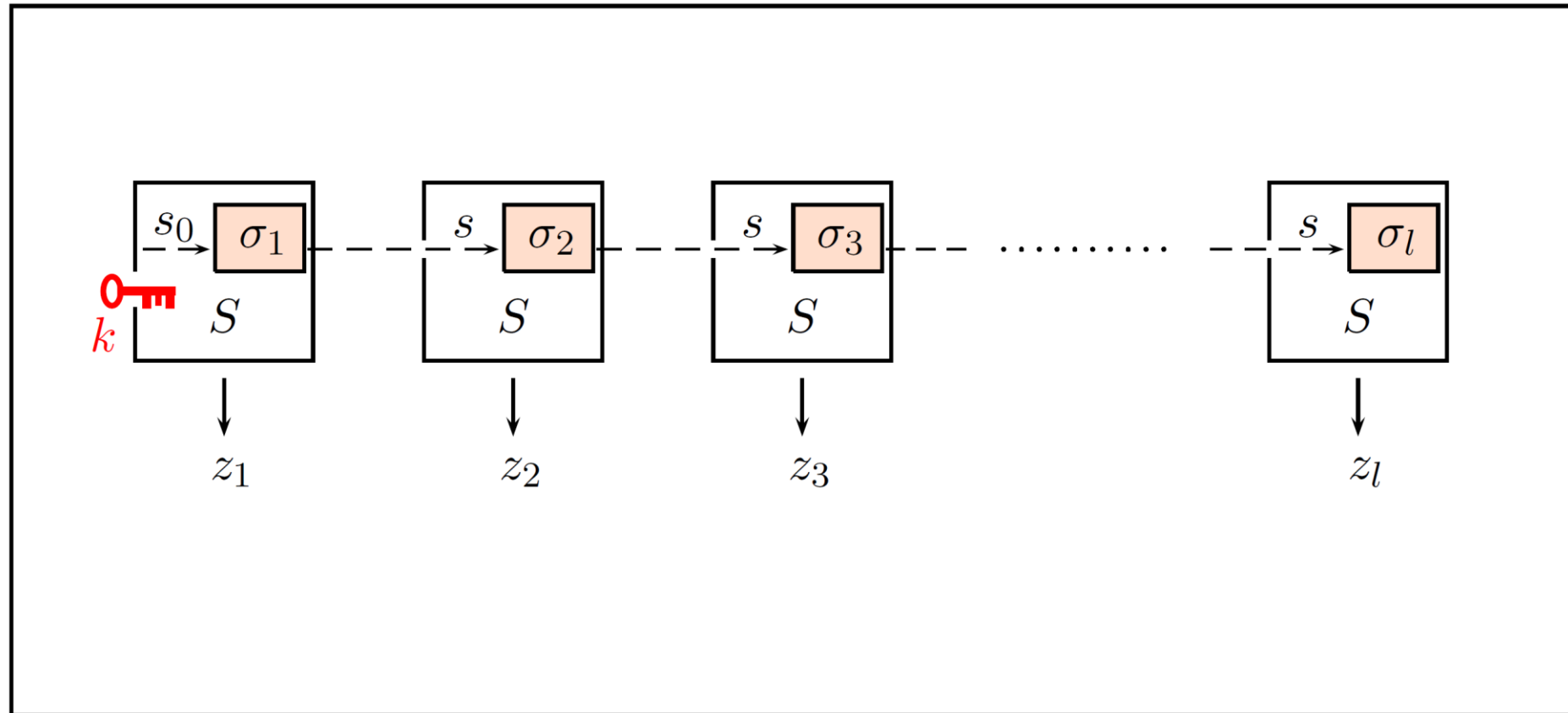


Abb. 1.6 Schlüsselstromgenerator eines synchronen additiven Stromverschlüsselungsverfahrens

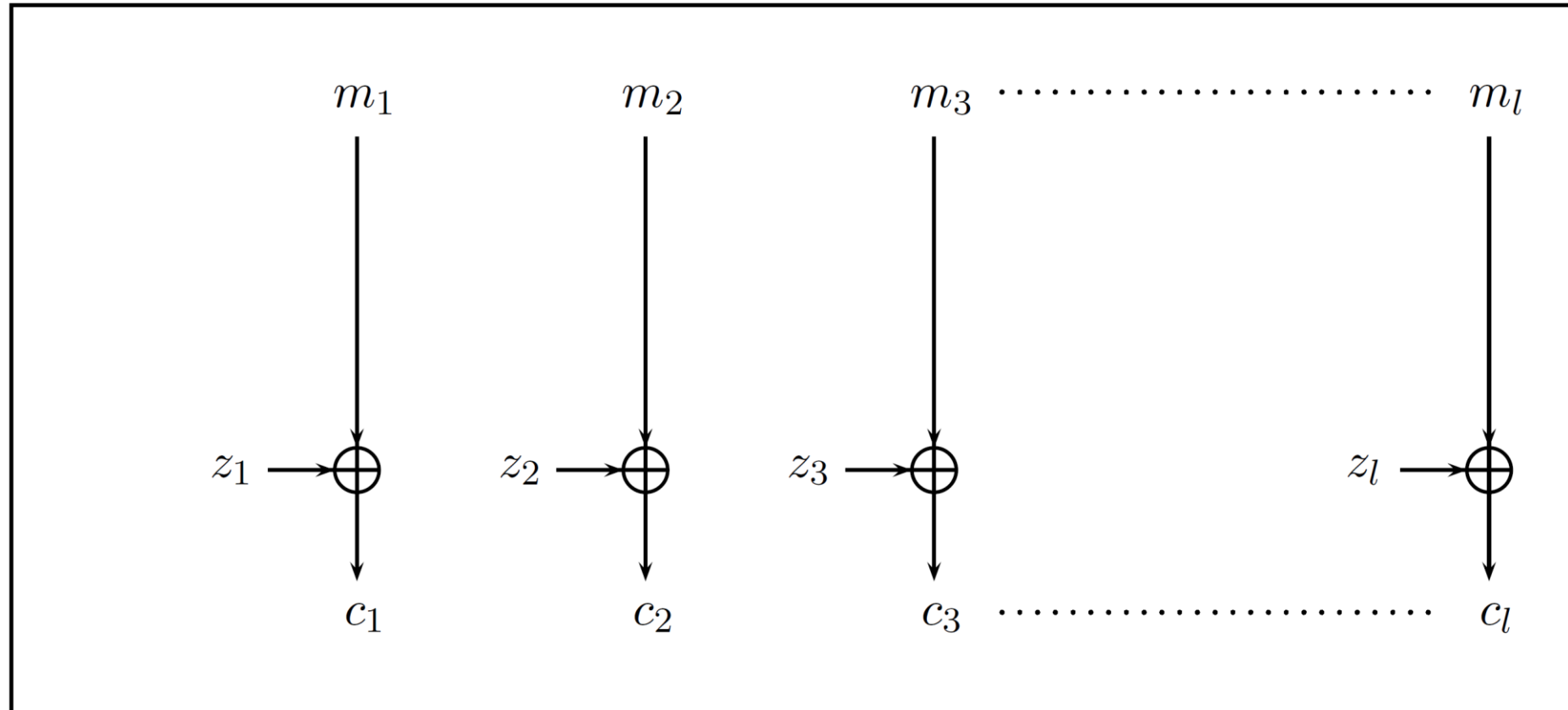


Abb. 1.7 Verschlüsselung der Klartextnachricht $(m_1, m_2, \dots, m_l)$ durch Addition mit dem Schlüsselstrom $(z_1, z_2, \dots, z_l)$

1.2 Schlüsselstrom vs. One-Time-Pad

Frage 2

Gegeben sei eine Ziffernmenge $\mathcal{Z}$, für deren Elemente eine Addition definiert ist. Geben Sie eine formale Beschreibung des One-Time-Pad-Verfahrens über $\mathcal{Z}$ für Nachrichten einer maximalen Länge $L \in \mathbb{N}$ an. Geben Sie hierzu eine geeigneten Menge $\mathcal{S}$ und geeignete Funktionen s_0 , s und S zur Beschreibung des Schlüsselstromgenerators an.

(Beachten Sie, dass der Schlüsselraum durch

$$\mathcal{K} = \mathcal{Z}^L$$

definiert ist.)

1.3 Nonces zur Initialisierung eines Schlüsselstromgenerators

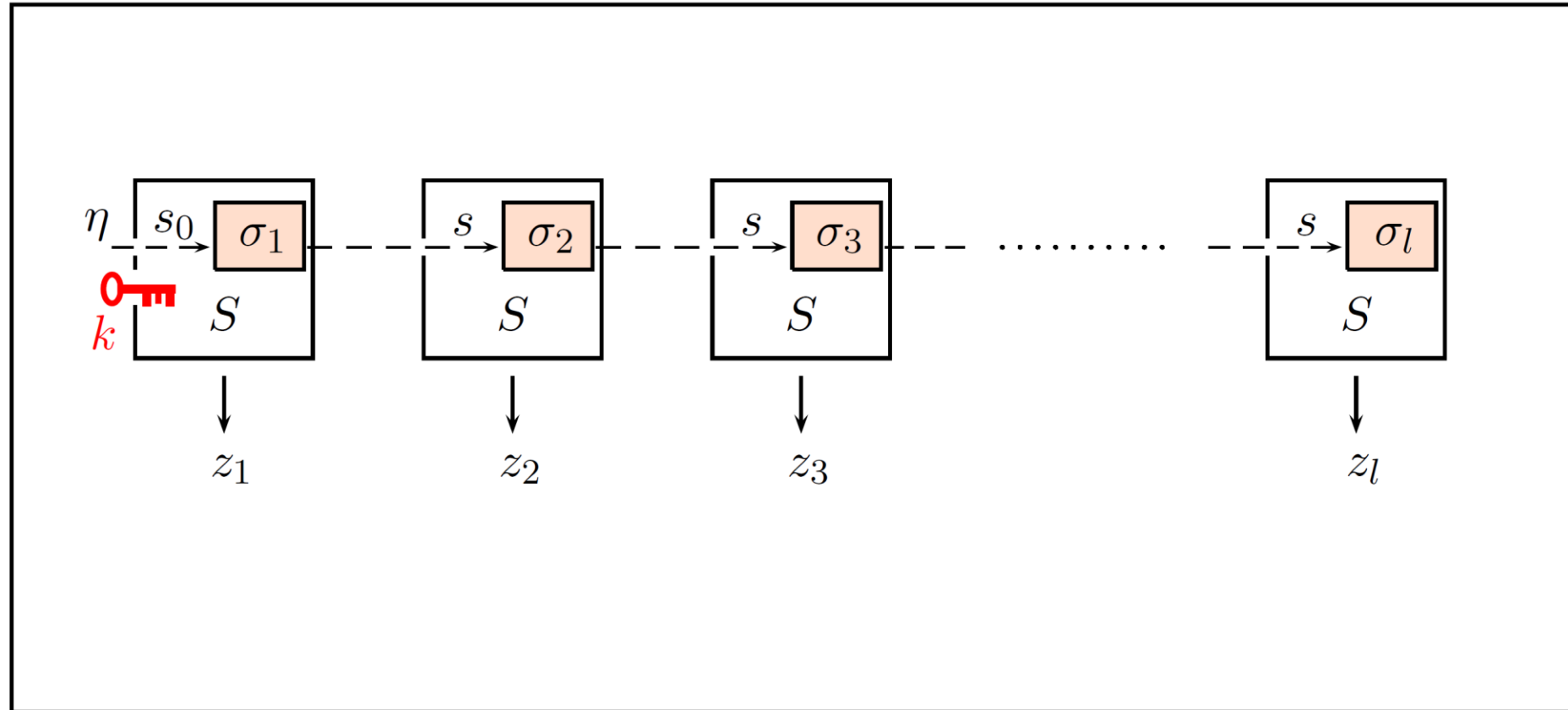


Abb. 1.8 Schlüsselstromgenerator, initialisiert mit Schlüsselwert k und Nonce-Wert η

$$\mathcal{Z} = \mathbb{Z}_{2^8}$$

Bytezahl

$$\mathcal{K} = (\mathbb{Z}_{2^{32}})^8$$

$$|\mathcal{K}| = 2^{256}$$

$$\mathcal{N} = (\mathbb{Z}_{2^{32}})^3$$

$$|\mathcal{N}| = 2^{96}$$

$$\mathcal{S} = (\mathbb{Z}_{2^{32}})^{16} \times (\mathbb{Z}_{2^{32}})^{16} \times \mathbb{Z}_{2^{38}+1}$$

Elemente des Zustandsraums $\mathcal{S}$ bestehen aus Tripeln:

$$(s, z, c) \in (\mathbb{Z}_{2^{32}})^{16} \times (\mathbb{Z}_{2^{32}})^{16} \times \mathbb{Z}_{2^{38}+1}$$

Ist $(k, \eta) = ((k_1, \dots, k_8), (n_1, n_2, n_3)) \in \mathcal{K} \times \mathcal{N}$ und $\underline{i \in \mathbb{Z}_{2^{32}}}$ gegeben, so ist

$$s^{\circledast i} := s^i(k, \eta) := (c_1, c_2, c_3, c_4, k_1, \dots, k_8, \circledast i n_1, n_2, n_3)$$

$$c_1 = 0x61707865$$

$$c_2 = 0x3320646e$$

$$c_3 = 0x79622d32$$

$$c_4 = 0x6b206574$$

Initialisierung: $s_0 : \mathcal{K} \times \mathcal{N} \rightarrow \mathcal{S}$

Sind $(k, \eta) \in \mathcal{K} \times \mathcal{N}$ so gilt mit $\mathbf{s}^0 = \mathbf{s}^0(k, \eta)$:

$$s_0((k, \eta)) = (\mathbf{s}^1, \mathbf{s}^0 +_{\mathbb{Z}_{2^{32}}} f_{\text{ib}}^{10}(\mathbf{s}^0), 0)$$

mit der nachfolgend definierten Funktion f_{ib} .

Update-Funktion: $s : \mathcal{S} \rightarrow \mathcal{S}$

Für $(\mathbf{s}^i, \mathbf{z}, c) \in \mathcal{S}$ (und $c < 2^{38}$) ist der Zustandsupdate wie folgt definiert:

$$s((\mathbf{s}^i, \mathbf{z}, c)) := \begin{cases} (\mathbf{s}^i, \mathbf{z}, c+1) & \text{falls } (c+1) \bmod 64 \neq 0 \\ (\mathbf{s}^{i+1}, \mathbf{s}^i +_{\mathbb{Z}_{2^{32}}} f_{\text{ib}}^{10}(\mathbf{s}^i), c+1) & \text{falls } (c+1) \bmod 64 = 0 \end{cases}$$

mit einer 10 mal wiederholt auszuführenden Funktion:

$$f_{\text{ib}} : (\mathbb{Z}_{2^{32}})^{16} \rightarrow (\mathbb{Z}_{2^{32}})^{16}$$

$$f_{\text{ib}} : (\mathbb{Z}_{2^{32}})^{16} \rightarrow (\mathbb{Z}_{2^{32}})^{16}$$

$$\begin{aligned}
 f_{\text{ib}}(\mathbf{s}) := & q(q(q(q(q(q(q(\mathbf{s}, 0, 4, 8, 12), \\
 & \quad 1, 5, 9, 13), \\
 & \quad 2, 6, 10, 14), \\
 & \quad 3, 7, 11, 15), \\
 & \quad 0, 5, 10, 15), \\
 & \quad 1, 6, 11, 12), \\
 & \quad 2, 7, 8, 13), \\
 & \quad 3, 4, 9, 14)
 \end{aligned}$$

Die Funktion

$$q : (\mathbb{Z}_{2^{32}})^{16} \times \mathbb{Z}_{16} \times \mathbb{Z}_{16} \times \mathbb{Z}_{16} \times \mathbb{Z}_{16} \rightarrow (\mathbb{Z}_{2^{32}})^{16}$$

bildet

$$(\mathbf{s}, i, j, k, l) = ((s_0, s_1, \dots, s_{15}), i, j, k, l)$$

auf das Element aus $(\mathbb{Z}_{2^{32}})^{16}$ ab, welches sich durch Abänderung der Komponenten

$$a = s_i, \quad b = s_j, \quad c = s_k, \quad d = s_l$$

$$a = s_i, \quad b = s_j, \quad c = s_k, \quad d = s_l$$

$$(a, b, c, d) \mapsto (a + b, b, c, d)$$

$$(a, b, c, d) \mapsto (a, b, c, d \oplus a)$$

$$(a, b, c, d) \mapsto (a, b, c, r_{16}(d))$$

$$(a, b, c, d) \mapsto (a, b, c + d, d)$$

$$(a, b, c, d) \mapsto (a, b \oplus c, c, d)$$

$$(a, b, c, d) \mapsto (a, r_{12}(b), c, d)$$

$$(a, b, c, d) \mapsto (a + b, b, c, d)$$

$$(a, b, c, d) \mapsto (a, b, c, d \oplus a)$$

$$(a, b, c, d) \mapsto (a, b, c, r_8(d))$$

$$(a, b, c, d) \mapsto (a, b, c + d, d)$$

$$(a, b, c, d) \mapsto (a, b \oplus c, c, d)$$

$$(a, b, c, d) \mapsto (a, r_7(b), c, d)$$

Extraktion der Schlüsselstromzeichen: $S : \mathcal{S} \rightarrow \mathcal{Z}$

Mit S werden die Bytewerte, aus denen die Zahlen $z_0, z_1, \dots, z_{15}$ der zweiten Komponente eines Elements aus $\mathcal{S}$ zusammengesetzt sind, extrahiert.

Für

$$(s^i, \mathbf{z}, c) = (s^i, (z_0, z_1, \dots, z_{15}), c) \in \mathcal{S}$$

werden hierzu

$$j = \lfloor c/4 \rfloor \bmod 16$$

$$k = c \bmod 4$$

berechnet und

$$S\left((s^i, (z_0, z_1, \dots, z_{15}), c)\right) := \left\lfloor \frac{z_j}{2^{8k}} \right\rfloor \bmod 2^8$$

gesetzt.

1.5 Cipher-Instanzen: Verchlüsselungsalgorithmen in Java-Laufzeitumgebungen

```
public static void test_JRE_RFC8439_2_4_2()
    throws NoSuchAlgorithmException, NoSuchPaddingException,
    InvalidKeyException, InvalidAlgorithmParameterException,
    IllegalBlockSizeException, BadPaddingException {

    byte[] key = new byte[32];
    for (byte i = 0; i < 32; ++i) {
        key[i] = i;
    }

    byte[] nonce
    = new byte[] {0, 0, 0, 0, 0, 0, 0, 0x4a, 0, 0, 0, 0};
    ChaCha20ParameterSpec params
    = new ChaCha20ParameterSpec(nonce, 1);

    Cipher cipher = Cipher.getInstance("ChaCha20");

    SecretKeySpec keyChaCha20
    = new SecretKeySpec(key, "ChaCha20");

    cipher.init(Cipher.ENCRYPT_MODE, keyChaCha20, params);

    byte[] m = ("Ladies and Gentlemen of the class of '99:"
    + " If I could offer you only one tip for the future,"
    + " sunscreen would be it.").getBytes();
    byte[] c = cipher.doFinal(m);
    System.out.println(Dump.dump(c));
    System.out.println();
}
```


Blockverschlüsselungsverfahren (Block Cipher)

2.1	Grundlagen	22
2.2	Padding-Verfahren	23
2.3	Betriebsmodi	23
2.4	Konstruktionsprinzipien von Blockverschlüsselungsver- fahren	35
2.5	DES und Triple-DES	36
2.6	Meet-in-the-Middle-Angriffe	38
2.7	AES	41

Blockverschlüsselungsverfahren (Block Cipher)

Ein Verschlüsselungsverfahren mit Verschlüsselungsfunktion E

$$E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$$

und Entschlüsselungsfunktion D

$$D : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$$

wird als *Blockverschlüsselungsverfahren* bezeichnet, falls die Menge der Nachrichten $\mathcal{M}$ durch die Menge der *Blöcke* einer festen Länge $n \in \mathbb{N}$ (der *Blocklänge* (in Bytes)) gegeben ist:

$$\mathcal{M} := (\mathbb{Z}_2^8)^n = \{(z_1, z_2, \dots, z_n) \mid z_i \in \mathbb{Z}_2^8\}$$



$$\mathcal{M} := (\mathbb{Z}_2^8)^n = \{(z_1, z_2, \dots, z_n) \mid z_i \in \mathbb{Z}_2^8\}$$

- Häufig werden Blöcke, die aus n Bytewerten bestehen als Blöcke aufgefasst, die aus $8 \cdot n$ Bitwerten bestehen:

$$(\mathbb{Z}_2^8)^n \longleftrightarrow \mathbb{Z}_2^{8 \cdot n}$$

Die Blocklänge (in Bits) wird dann mit $8 \cdot n$ angegeben.

- Mit der obigen Identifikation kann die Menge der Blöcke $\mathcal{M}$ als Vektorraum über $\mathbb{Z}_2$ aufgefasst werden und für zwei Blöcke $m_1, m_2 \in \mathcal{M}$ ist die Summe

$$m_1 + m_2$$

durch eine bitweise Addition definiert.

XOR-Verknüpfung

$$\oplus$$

- Da die Vektoren aus $\mathbb{Z}_2^{8 \cdot n}$ den Binärdarstellungen der Zahlen aus $\mathbb{Z}_{2^{8 \cdot n}}$ eindeutig entsprechen, könnte eine Addition von zwei Blöcke auch vermittels der Addition in $\mathbb{Z}_{2^{8 \cdot n}}$ definiert werden.

$$\overline{m \boxplus z} = (\overline{m} + z) \bmod 2^{8 \cdot n}$$

$$m \boxplus z$$

Für jeden Schlüssel $k \in \mathcal{K}$ ist E_k eine Permutation der Menge der Blöcke $\mathcal{M}$ mit Umkehrfunktion D_k . Bei einem guten Blockverschlüsselungsverfahren sollte hierbei gelten:

Die Abbildung

$$\mathcal{K} \rightarrow \Sigma_{\mathcal{M}}, \quad k \mapsto E_k$$

lässt sich mit statistischen Verfahren nicht von einer Zuordnung der Schlüsselwerte zu zufällig ausgewählten Permutationen aus $\Sigma_{\mathcal{M}}$ unterscheiden.

2.2 Padding-Verfahren

Bsp. Blocklänge $n = 16$

H I L L O X - - - - -

|
Erkennung?

- ECB (Electronic Code Book)
- CBC (Cipher Block Chaining)
- CTS (Ciphertext Stealing)
- CTR (Counter)
- OFB (Output Feedback)
- CFB (Cipher Feedback)

■ GCM - Modus (inkl. MAC-Berechnung) (spezifiziert in TLS)

Betriebsmodus ECB (Electronic Code Book)

Im *ECB-Modus* wird mit einem Blockverschlüsselungsverfahren E jedes Tupel

$$(m_1, \dots, m_r)$$

von Blöcken $m_i \in \mathcal{M}$ für einen gegebenen Schlüssel $k \in \mathcal{K}$ blockweise verschlüsselt:

$$E_k((m_1, \dots, m_r)) := (E_k(m_1), \dots, E_k(m_r))$$

Diese einfache komponentenweise Verschlüsselung ist in Abbildung 2.1 dargestellt.

Zur Entschlüsselung ist die Entschlüsselungsfunktion D ebenfalls einfach komponentenweise anzuwenden:

$$D_k((c_1, \dots, c_r)) := (D_k(c_1), \dots, D_k(c_r))$$

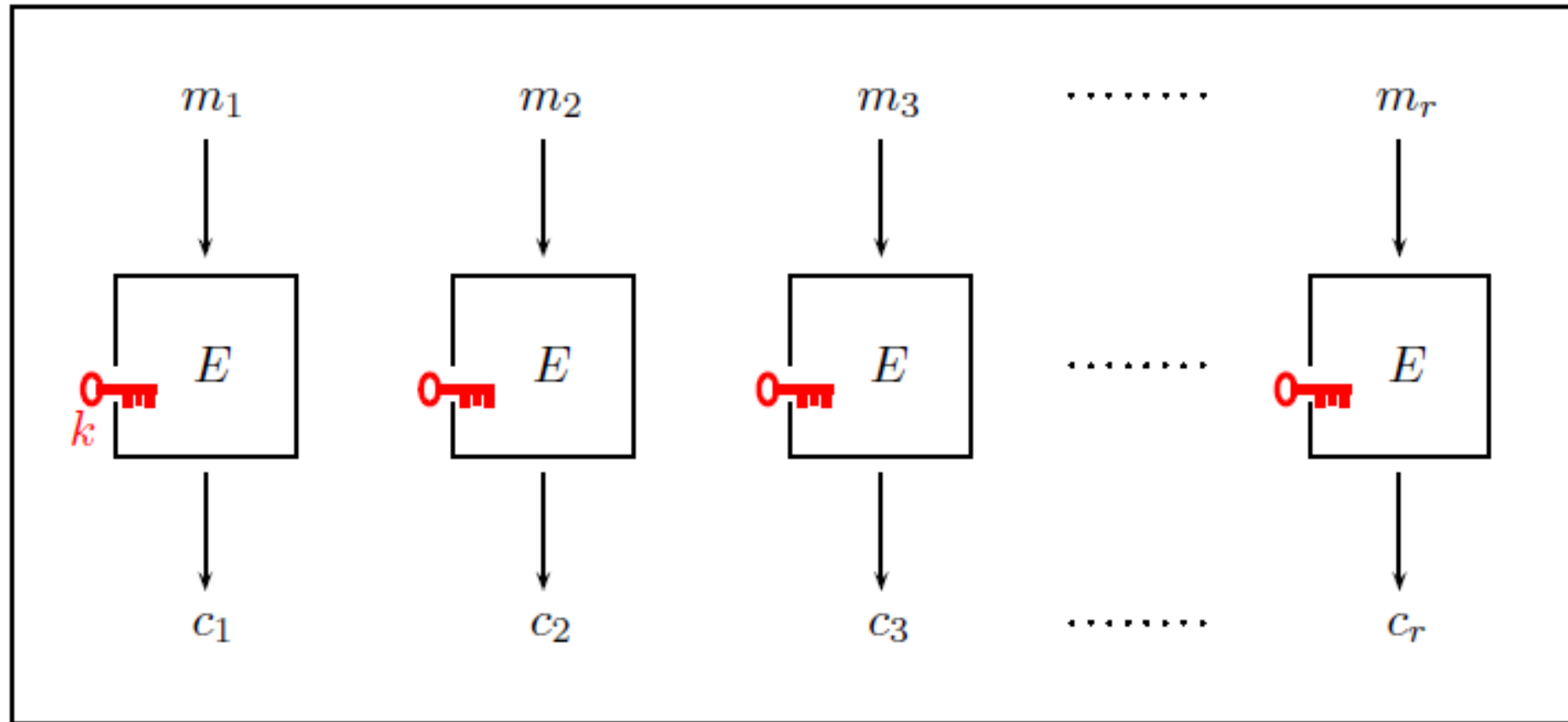


Abb. 2.1 Blockverschlüsselungsverfahren im ECB-Modus

Frage 4

Vergleichen Sie Abbildung 2.1 mit Abbildung 1.2 und bringen Sie die jeweils dargestellten Verfahren in einen Zusammenhang.


```
public static void main(String[] args) throws Exception {  
    byte[] key = Dump.hexString2byteArray(  
        "0102030405060708090A0B0C0D0E0F10");  
    SecretKey secretKey = new SecretKeySpec(key, "AES");  
  
    Cipher cipher  
        = Cipher.getInstance("AES/ECB/NOPADDING");  
  
    cipher.init(Cipher.ENCRYPT_MODE, secretKey);  
  
    byte[] m = new byte[48];  
    m[0] = m[32] = 'T';  
    m[1] = m[33] = 'e';  
    m[2] = m[34] = 's';  
    m[3] = m[35] = 't';  
    System.out.println("Klartext:");  
    System.out.println(Dump.dump(m));  
  
    byte[] c = cipher.doFinal(m);  
  
    System.out.println("Geheimtext:");  
    System.out.println(Dump.dump(c));  
}
```

Aufgabe 3

Führen Sie die Methode aus Abbildung 2.2 zur Verschlüsselung einer 48 Byte langen Nachricht aus. Beachten Sie, dass nach der Instanziierung des Cipher-Objekts mit "AES/ECB/NOPADDING" dieses Objekt eine Implementierung des AES-Algorithmus im ECB-Modus zur Verfügung stellt, bei der die Daten nicht mit einem Padding ergänzt werden, so dass die Datenlänge ein Vielfaches der Blocklänge sein muss! (In dem Beispiel ist dies erfüllt, da beim AES-Verfahren die Blöcke 16 Byte lang sind.)

Beachten Sie, dass der erste und dritte Klartextblock und entsprechend auch der erste und dritte Geheimtextblock gleich sind.

Erweitern Sie das Programm, so dass der in der in c gespeicherte Geheimtext wieder entschlüsselt wird.

Aufgabe 4

Verschlüsseln Sie die nur aus den ersten 4 Byte von m aus Abbildung 2.2 bestehende Nachricht mit dem AES-Verfahren, indem Sie eines der unterstützten Padding-Verfahren (PKCS5PADDING, ISO10126PADDING) benutzen.

Ersetzen Sie hierzu beispielsweise die Zeilen 6–15 aus Abbildung 2.2 durch den Quellcode aus Abbildung 2.3.

Frage 5

Untersuchen Sie die unterstützten Paddingverfahren PKCS5PADDING und ISO10126PADDING indem Sie mehrfach eine Verschlüsselung mit diesen Verfahren durchführen und die Geheime-texte anschließend mit der Option NOPADDING entschlüsseln, so dass das Padding nach der Entschlüsselung zusammen mit dem Klartext sichtbar wird.

Wie unterscheiden sich die Padding-Verfahren PKCS5PADDING und ISO10126PADDING?

s. hier - Testaufgaben

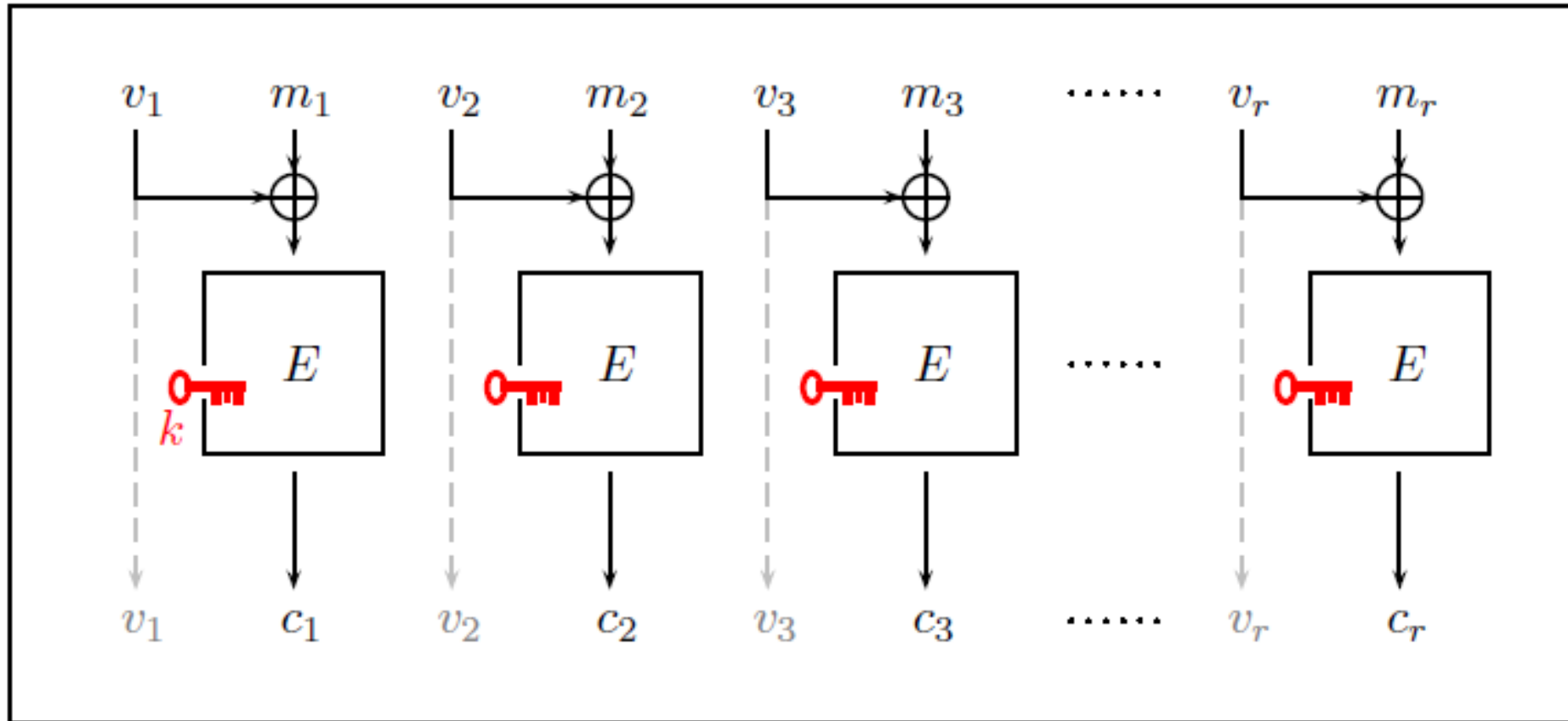


Abb. 2.4 Blockverschlüsselungsverfahren mit Nonce-Werten v_i

Betriebsmodus CBC (Cipher Block Chaining)

Im *CBC-Modus* wird mit einem Blockverschlüsselungsverfahren E jedes Tupel

$$(m_1, \dots, m_r)$$

von Blöcken $m_i \in \mathcal{M}$ für einen gegebenen Schlüssel $k \in \mathcal{K}$ und einen Nonce-Wert

$$IV \in \mathcal{M}$$

folgendermaßen rekursiv blockweise verschlüsselt:

$$c_0 := IV$$

$$c_i := E_k(m_i + c_{i-1}) \quad \text{für } i = 1, \dots, r$$

$$E_{k,IV}(m_1, \dots, m_r) := (c_1, \dots, c_r)$$

Die Verschlüsselung im CBC-Modus ist in Abbildung 2.5 dargestellt.

Die Entschlüsselung eines CBC-verschlüsselten Geheimtextes $c = (c_1, \dots, c_r)$ ist durch

$$c_0 := IV$$

$$m_i := D_k(c_i) + c_{i-1} \quad \text{für } i = 1, \dots, r$$

$$D_{k,IV}(c_1, \dots, c_r) := (m_1, \dots, m_r)$$

gegeben. Die Verschlüsselung im CBC-Modus ist in Abbildung 2.6 dargestellt.

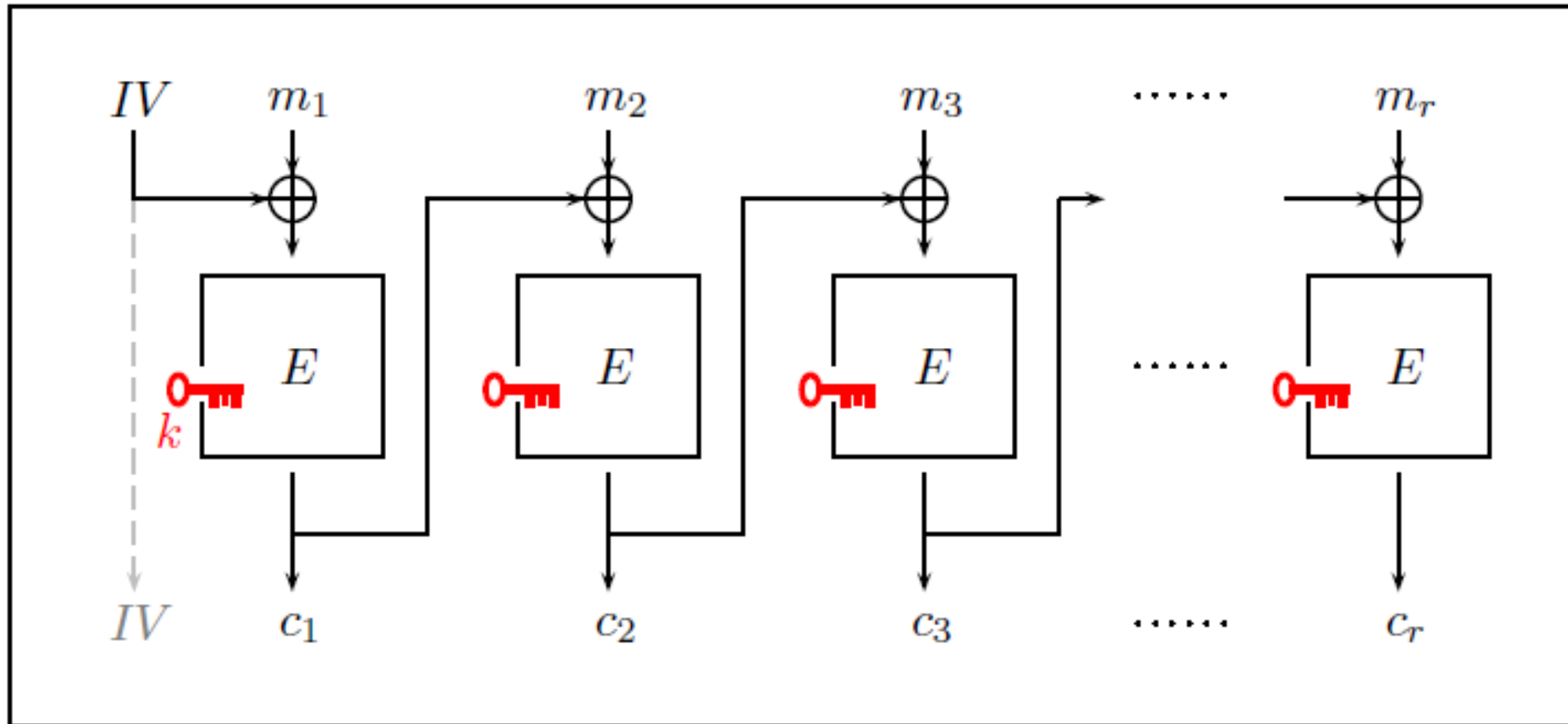


Abb. 2.5 Verschlüsselung mit Blockverschlüsselungsverfahren im CBC-Modus

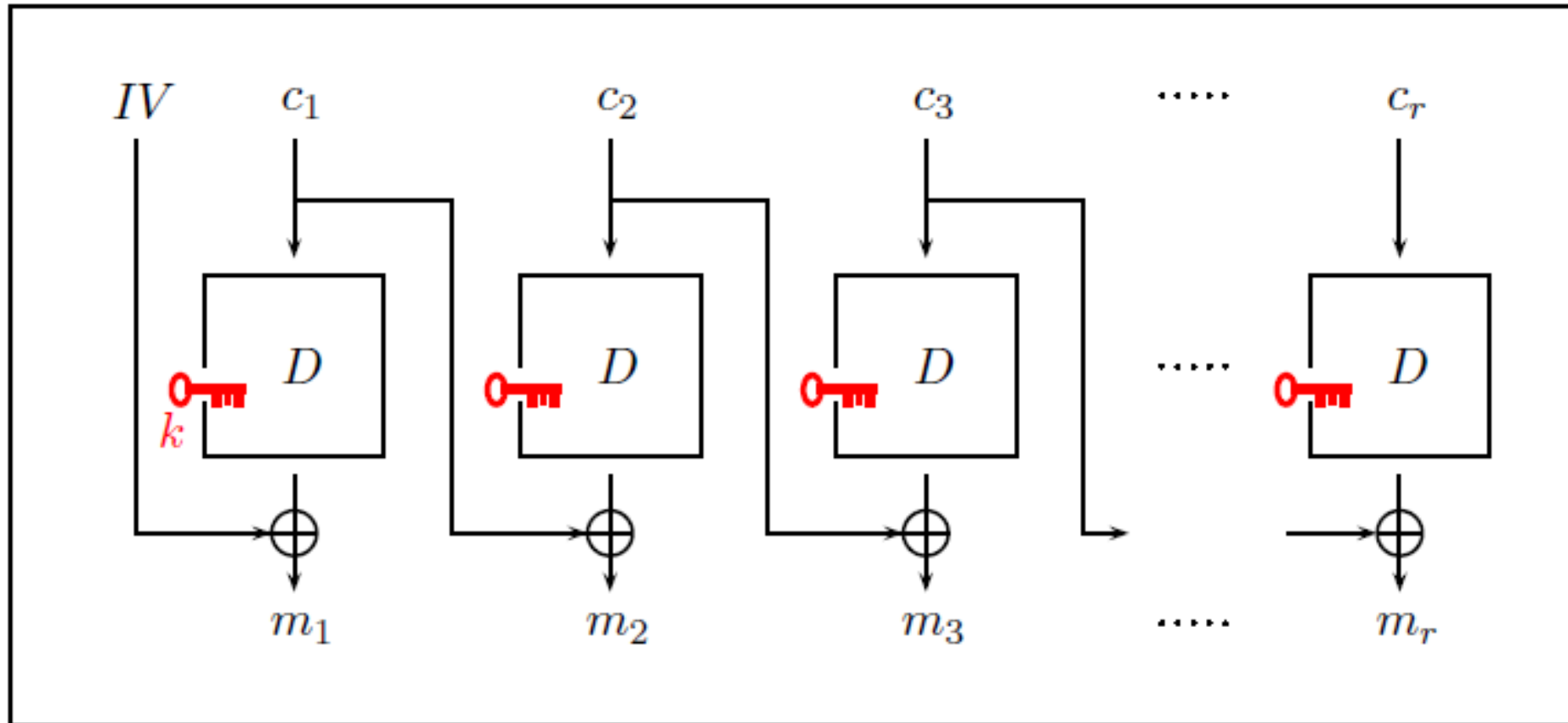


Abb. 2.6 Entschlüsselung mit einem Blockverschlüsselungsverfahren im CBC-Modus

Aufgabe 5

Ändern Sie den Quellcode aus Abbildung 2.2 so ab, dass anstelle des ECB-Modus ohne Padding der CBC-Modus mit dem PKCS5PADDING-Verfahren zum Einsatz kommt.

Ersetzen Sie hierzu beispielsweise die Zeilen 6–9 aus Abbildung 2.2 durch den Quellcode aus Abbildung 2.7. (IV-Werte müssen durch Instanzen der Klasse `IvParameterSpec` gekapselt werden, um sie einer geeigneten `init(...)`-Methode als Parameter übergeben zu können.)

Beachten Sie, dass der erste und dritte Geheimtextblock nun verschieden sind, obwohl erster und dritter Klartextblock identisch sind.

Erweitern Sie das Programm, so dass der in der in `c` gespeicherte Geheimtext wieder entschlüsselt wird.

```

6      Cipher cipher
      = Cipher.getInstance("AES/CBC/PKCS5PADDING");

      byte[] ivBytes = new byte[16];
10     new SecureRandom().nextBytes(ivBytes);
      IvParameterSpec iv = new IvParameterSpec(ivBytes);
      cipher.init(Cipher.ENCRYPT_MODE, secretKey, iv);
  
```

Abb. 2.7 Änderungen des Quellcodes aus Abbildung 2.2 zur Verschlüsselung im CBC-Modus

Frage 6

Alice verschlüsselt die Nachricht $(m_1, m_2, \dots, m_9)$ mit dem CBC-Verfahren unter Nutzung des Initialisierungswerts IV . Bei der Übertragung von IV und $(c_1, c_2, \dots, c_9)$ an Bob wird ein Bit des Blocks c_2 fehlerhaft empfangen.

Welche Teile der ursprünglichen Klartextnachricht kann Bob fehlerfrei lesen?

hier - Testaufgabe

Frage 7

Lässt sich die Verschlüsselung und/oder Entschlüsselung im CBC-Modus synchron blockweise durchführen?

Betriebsmodus CBC-CS1 (Ciphertext Stealing for CBC, Variante 1)

Gegeben sei ein Blockverschlüsselungsverfahren E für Blöcke der Byte-Länge n , ein Schlüssel $k \in \mathcal{K}$ und ein Nonce-Wert $IV \in \mathcal{M}$.

Nachrichten, die mindestens n Bytes umfassen, können in der Form

$$m = (m_1, \dots, m_{r-1}, m_r^*)$$

mit Byte-Arrays

$$m_1, \dots, m_{r-1} \in \mathcal{M} \quad \text{und} \quad l = l(m_r^*) \in \{1, \dots, n\}$$

geschrieben werden, wobei $l(m_r^*)$ die Byte-Länge von m_r^* bezeichnet.

Es sei

$$m_r \in \mathcal{M}$$

der Block, der aus m_r^* durch das Anfügen von $n - l$ vielen 00-Bytes hervorgeht (Padding von m_r^* mit 00 ... 00 zu m_r). Gilt $l = n$, so ist $m_r = m_r^*$.

Weiterhin sei

$$(c_1, \dots, c_r) = E_{k, IV}(m_1, \dots, m_r)$$

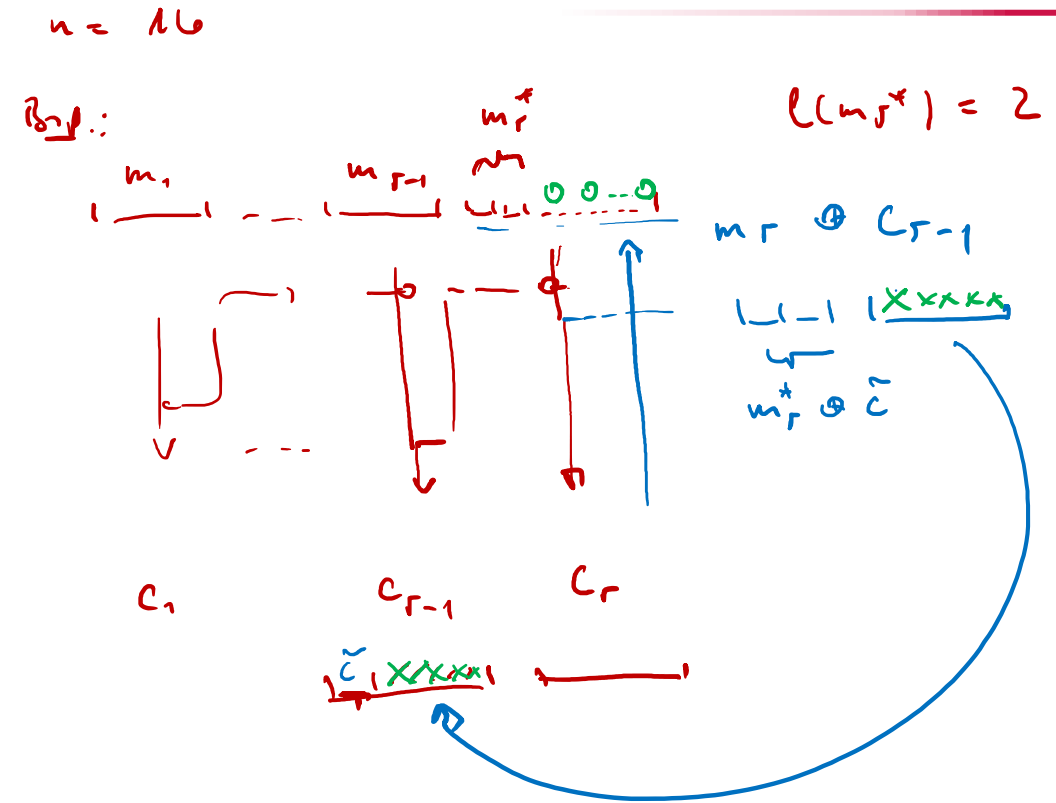
die Verschlüsselung von $(m_1, \dots, m_r)$ im CBC-Modus.

Die Verschlüsselung im CBC-CS1-Modus von m ist schließlich definiert durch

$$E\text{-CBC-CS1}_{k, IV}(m_1, \dots, m_{r-1}, m_r^*) := (c_1, \dots, c_{r-2}, c_{r-1}^*, c_r)$$

wobei c_{r-1}^* nur aus den ersten l Bytes von c_{r-1} besteht.

Beachten Sie, dass die Geheimtextnachricht $(c_1, \dots, c_{r-2}, c_{r-1}^*, c_r)$ die gleiche Länge wie die Klartextnachricht m besitzt.



Kommentar

- (i) Die Vorgehensweise beim CBC-CS1-Modus lässt sich kurz so zusammenfassen, dass die Klartextnachricht mit einer minimalen Anzahl p von 00-Bytes auf ein Vielfaches der Blocklänge gebracht wird (Padding), dann im CBC-Modus verschlüsselt wird und vom vorletzten Geheimtextblock die letzten p Bytes entfernt („gestohlen“) werden.
- (ii) Ist die Länge der Klartextnachricht ein Vielfaches der Blocklänge, so entspricht eine Verschlüsselung im CBC-CS1-Modus einer Verschlüsselung im CBC-Modus.

Frage 8

Beschreiben Sie, wie eine Geheimtextnachricht c im CBC-CS1-Modus zu entschlüsseln ist.

Betriebsmodus CBC-CS2 (Ciphertext Stealing for CBC, Variante 2)

Ebenso wie beim CBC-CS1-Modus werden Klartextnachrichten, deren Länge ein Vielfaches der Blocklänge n ist, einfach ohne weitere Veränderung im CBC-Modus verschlüsselt.

Die Verschlüsselung einer Klartextnachricht, deren Länge $> n$ und kein Vielfaches von n ist, wird im CBC-CS2-Modus grundsätzlich genauso wie im CBC-CS1-Modus berechnet, nur dass zum Abschluss die beiden letzten Tupel der Geheimtextnachricht vertauscht werden. Mit den Bezeichnungen aus der Beschreibung des CBC-CS1-Modus gilt also in diesem Fall:

$$E\text{-CBC-CS2}_{k,IV}(m_1, \dots, m_{r-1}, m_r^*) := (c_1, \dots, c_{r-2}, c_r, c_{r-1}^*)$$

Betriebsmodus CBC-CS3 (Ciphertext Stealing for CBC, Variante 3)

Die Verschlüsselung einer Klartextnachricht wird im CBC-CS3-Modus wiederum zunächst wie im CBC-CS1-Modus berechnet, nur dass danach noch die beiden letzten Tupel der Geheimtextnachricht immer vertauscht werden (also auch dann, falls die Länge der Klartextnachricht ein Vielfaches der Blocklänge ist).

Frage 9

Eine Implementierung einer der drei oben beschriebenen CBC-CS-Varianten steht in Java für AES nach einer Initialisierung der Cipher-Klasse mit "AES/CTS/NOPADDING" zur Verfügung. Bestimmen Sie mit einem kleinen Testprogramm, welche Variante dies ist.

1. Test

Betriebsmodus CTR (Counter)

Im *CTR-Modus* wird mit einem Blockverschlüsselungsverfahren E jedes Tupel

$$(m_1, \dots, m_r)$$

von Blöcken $m_i \in \mathcal{M}$ für einen gegebenen Schlüssel $k \in \mathcal{K}$ und einen Nonce-Wert

$$Ctr \in \mathcal{M}$$

folgendermaßen blockweise verschlüsselt:

$$c_i := m_i + E_k(Ctr \boxplus (i-1)) \quad \text{für } i = 1, \dots, r$$

$$E\text{-CTR}_{k,Ctr}(m_1, \dots, m_r) := (c_1, \dots, c_r)$$

Die Verschlüsselung im CTR-Modus ist in Abbildung 2.8 dargestellt.

Der CTR-Modus definiert ein synchrones additives Stromverschlüsselungsverfahren, dessen Schlüsselstrom aus den Bytewerten der Blöcke

$$z_i = E_k(Ctr \boxplus (i-1)) \quad \text{für } i = 1, 2, \dots$$

besteht.

Die Entschlüsselungsfunktion ist daher identisch zur Verschlüsselungsfunktion:

$$D\text{-CTR}_{k,Ctr}(c_1, \dots, c_r) = E\text{-CTR}_{k,Ctr}(c_1, \dots, c_r)$$

Schlüsselstrom generator

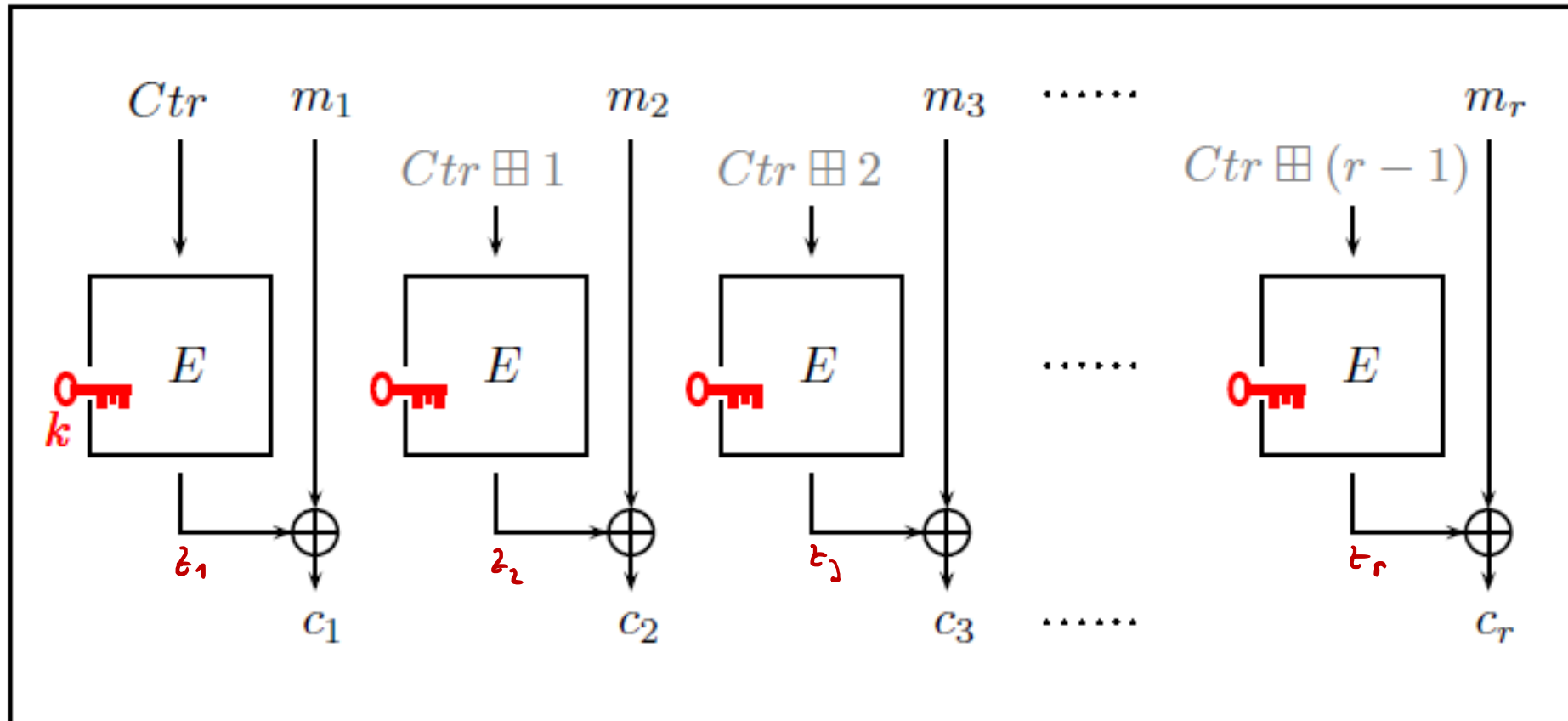


Abb. 2.8 Blockverschlüsselungsverfahren im Counter-Modus

Frage 10

Sind $l, n \in \mathbb{N}$, so lässt sich l auf eine eindeutig bestimmte Weise als Summe eines ganzzahligen Vielfachen von n und einer positiven Zahl $\leq n$ schreiben. D. h., es gibt eindeutig bestimmte v, w mit:

$$l = v \cdot n + w, \quad v \in \mathbb{Z}, \quad w \in \{1, 2, \dots, n\}$$

Zeigen Sie:

$$v = \lfloor (l-1)/n \rfloor$$

$$w = ((l-1) \bmod n) + 1$$

Frage 11

Schreiben Sie den Quellcode für eine Klasse `KeyStreamGenerator_AES_CTR`, die die Klasse `KeyStreamGenerator` aus Abbildung 1.10 erweitert und den durch den CTR-Modus definierten Schlüsselstromgenerator auf Basis des AES-Blockverfahrens (Cipher-Instanz vom Typ "AES/ECB/NOPADDING") implementiert.

Aufgabe 6

Testen Sie Ihre Lösung zu Frage 11 indem Sie verschiedene Klartextnachrichten (unterschiedlicher Längen) für verschiedene Ctr-Werte mit der von Ihnen implementierten Methode und zum Vergleich mit einer Cipher-Instanz vom Typ "AES/CTR/NOPADDING" verschlüsseln.

Eine Methode für einen solchen Vergleich findet sich beispielhaft in Abbildung 2.9.

```

1  public static void main(String[] args) throws Exception {
    byte[] key = Dump.hexString2byteArray(
        "0102030405060708090A0B0C0D0E0F10");
    SecretKey secretKey = new SecretKeySpec(key, "AES");

5

    Cipher cipher
        = Cipher.getInstance("AES/CTR/NOPADDING");

    byte[] ctr = Dump.hexString2byteArray(
10     "FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE");
    IvParameterSpec iv = new IvParameterSpec(ctr);
    cipher.init(Cipher.ENCRYPT_MODE, secretKey, iv);

    byte[] m = new byte[35];

15

    byte[] c = cipher.doFinal(m);
    System.out.println("AES-CTR-Verschlüsselung mit "
        + "Cipher-Instanz vom Typ AES/CTR/NOPADDING:");
    System.out.println(Dump.dump(c));
20    System.out.println();

    KeyStreamGenerator_AES_CTR ksg
        = new KeyStreamGenerator_AES_CTR();
    ksg.initState(key, ctr);
25    AdditiveStreamCipher.encrypt(ksg, m);
    System.out.println("AES-CTR-Verschlüsselung mit "
        + "KeyStreamGenerator_AES_CTR-Implementierung:");
    System.out.println(Dump.dump(m));
    System.out.println();
30 }
  
```

Abb. 2.9 AES-CTR-Verschlüsselung mit einer Cipher-Instanz vom Typ "AES/CTR/NOPADDING" und mit der Implementierung KeyStreamGenerator_AES_CTR aus Frage 11

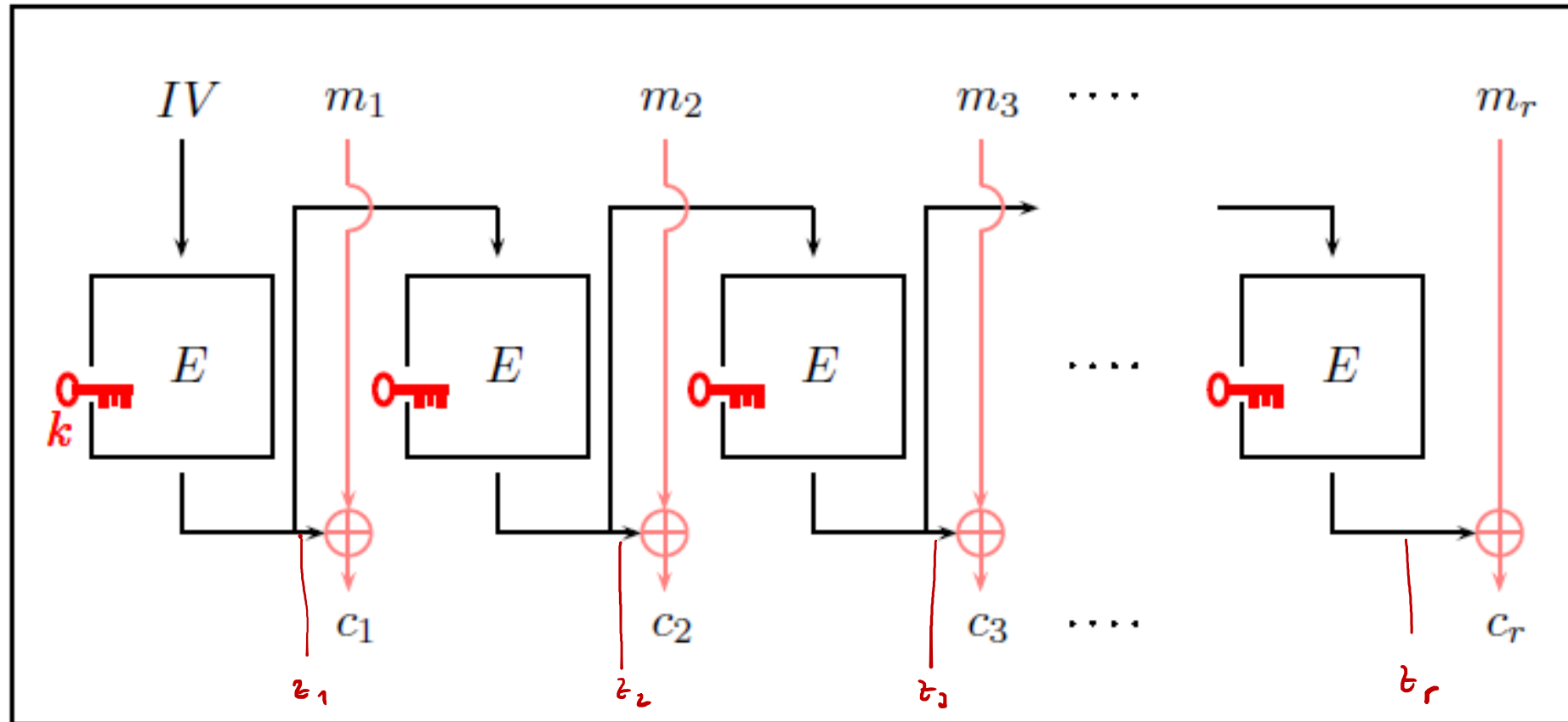


Abb. 2.10 Blockverschlüsselungsverfahren im OFB-Modus

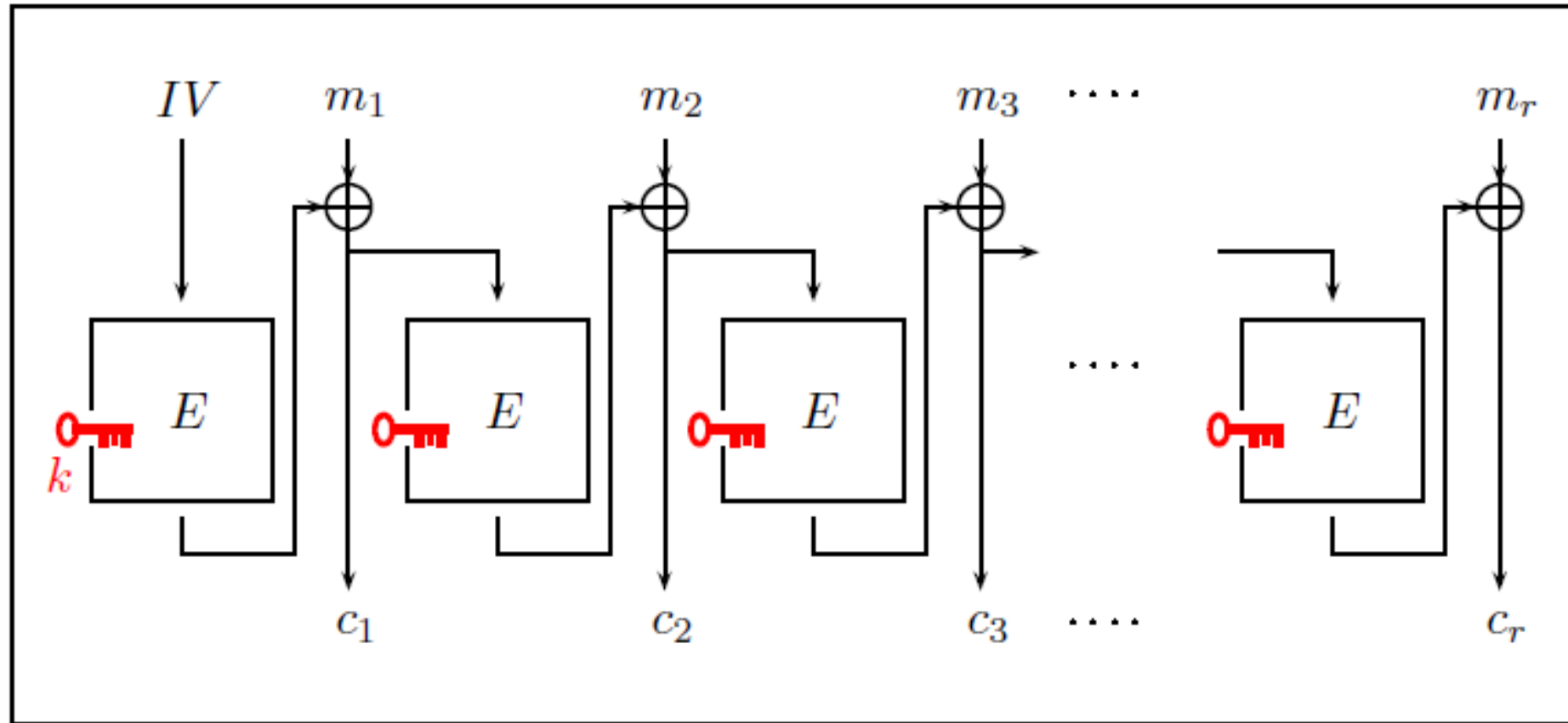


Abb. 2.12 Blockverschlüsselungsverfahren im CFB-Modus